

The Harness–Capability Benchmark

A pre-registered 2×2 factorial crossing harness design
with model deployment in personal AI assistants

Marc Cholevas

September 14, 2026

Abstract

Personal AI assistants that run on a user's own device keep every request private, and pay for it in capability: a model small enough to run on a laptop is meaningfully weaker than one running on a company's servers. A builder who has ruled out the cloud cannot send the hard requests elsewhere, so the remaining question is whether the harness — the program around the model that decides how a request is handled — can make up the difference. Published work points both ways.

This experiment crosses the two factors directly. Two harness designs were compared: routing, where the model names one action and whatever that action returns is the reply the user reads, and deciding, where the model sees each result and may act again. Each was run at two deployments, cloud and on-device, over 48 tasks in six categories. Hypotheses, measures, thresholds and analysis were fixed in writing before any data existed, in two studies.

The deciding design does not compensate for a weaker model. Over 960 runs it is worth +0.179 in task success at cloud deployment and +0.133 on the device, and the difference between those is -0.046 , with a 95% interval of $[-0.196, +0.100]$. Across 12 pairings of six on-device models against two cloud models over 2,688 runs — the first study's 960 and 1,728 more, the ablation counted separately — 11 contrasts are negative, three of them separating from zero, and the twelfth does not. An ablation whose direction was registered before it ran locates the design's entire advantage in one property: removing the loop that returns an action's result to the model removes the advantage at both deployments.

One result was not anticipated in the second study. Under the routing design a stronger of the two cloud models scored worse than a weaker one, because it chose to act more often — and under routing, acting means the reply the user reads is written by the action rather than the model. For a builder, the harness is worth building and the loop is the part that matters, but the capability a device costs is a cost to plan for, not one to design around.

Keywords: on-device deployment; harness design; pre-registration; ablation; task success; compensation; factorial; personal AI assistants

Commitments on Academic Honesty and Integrity

The participating team declares that the paper submitted is comprised of original research and results obtained under the guidance of the instructor. To the team's best knowledge, the paper does not contain research results, published or not, from a person who is not a team member, except for the content listed in the references and the acknowledgment. If there is any misinformation, we are willing to take all the related responsibilities.

Names of team member(s):

[withheld]

Signatures of team member(s):

[withheld]

Name of the instructor:

[withheld]

Signature of the instructor:

[withheld]

Date:

September 14, 2026

Table of Contents

Abstract.....	2
Commitments on Academic Honesty and Integrity	3
Table of Contents	4
§1 Introduction.....	6
§1.1 A request that needs a decision	6
§1.2 Two ways to build the decision mechanism	7
§1.3 Why the answer is not obvious	8
§1.4 The question, the design, and what was found.....	9
§1.5 Contributions.....	12
§2 Related Work	13
§2.1 Running the model on the user's device	13
§2.2 Two established designs	14
§2.3 The evidence points both ways	14
§2.4 The two axes have not been crossed	15
§3 Method	16
§3.1 The two factors, and what was held constant.....	16
§3.2 The two designs	18
§3.3 The two models.....	21
§3.4 The instrument	22
§3.5 The tasks	24
§3.6 What was measured	25
§3.7 Who decided what counted as correct.....	26
§3.8 How the run was carried out	32
§3.9 How the numbers were computed.....	34
§3.10 The plan, and every departure from it.....	37
§3.11 The criterion that was not met.....	38
§3.12 Threats identified in advance	40
§4 Results	42
§4.1 What the 960 runs contain	42
§4.2 The headline: no detectable compensation	42
§4.3 The two main effects.....	44

§4.4 How much to trust these numbers.....	45
§4.5 What the deciding design does differently.....	48
§4.6 By category	51
§4.7 The criterion in the full run.....	52
§4.8 What the advantage costs.....	53
§5 The second study	55
§5.1 What the first study could not answer.....	55
§5.2 What was registered, and when.....	55
§5.3 What was added	56
§5.4 Compensation across 12 pairings.....	58
§5.5 Where the advantage comes from.....	62
§5.6 What the second study does not settle	63
§6 Discussion.....	65
§6.1 What the harness buys back.....	65
§6.2 If only one thing could be built.....	65
§6.3 The routing design penalizes its own model.....	66
§6.4 What is settled, and what remains open	66
§7 What this study does not show.....	68
§8 Conclusion	71
Acknowledgements	72
Declaration of assistance from artificial intelligence.....	72
References.....	74
Appendix A — The task set.....	76
Appendix B — The action set	84
Appendix C — The pre-registration, and the full record of departures from it.....	87
C.1 The pre-registration, and where it is.....	87
C.2 Amendments — before any data existed.....	88
C.3 Deviations — after data existed	89
Study 2 — three more	90
C.4 Where the plan and the repository disagree	91
C.5 The 14 pre-registered threats, and where each is answered	91
C.6 The second rater's readings of action matching.....	93
Appendix D — Conversational naturalness	95

§1 Introduction

§1.1 A request that needs a decision

Personal AI assistants are increasingly built to run on a user's own device rather than on a company's servers. The reasons are practical: requests stay private, there is no per-use fee, and the assistant works without an internet connection. The cost is capability. A model small enough to run on a laptop is meaningfully weaker than one running in a data center, and the builder cannot have both. §2.1 reports what published comparisons measure that gap to be. What that builder can still change is how the assistant is built around the model. Whether a better harness recovers what the weaker model loses has not been measured: both decisions have been separately studied and never crossed (§2.4).

“Clear out the old downloads, they're taking up space” is a request someone could make to their assistant. “Old” is not specified here — does this refer to a week old, a month old, or anything never opened before? The assistant must choose between asking the user for clarification and committing to an interpretation. If the assistant picks incorrectly and acts, wanted files are permanently deleted. The request needs a decision before it needs an action. Ordinary instructions like this one are frequently underspecified. The underspecification is a trait of how people speak, not a defect in this particular request, and it makes a decision unavoidable.

For the purpose of this study, an assistant is treated as consisting of two components: an underlying model and a harness. The model provides capabilities such as reasoning and language understanding. Builders do not create it; they either download it or buy access to it. The harness is the surrounding infrastructure that decides what the model is shown, how the model interacts with the world, whether it can see the result of its actions, whether it can revise its decisions, and how its output is presented to the user. The harness is what the builder constructs. Builders face two major design decisions: where to run the model, and how to design the harness.

Models can be run on a company's servers or on the user's own device. However, the two options are not the same model in two places. Models small enough to run on most devices, such as a laptop or a phone, are meaningfully weaker than the frontier models that run on corporate servers. Someone who needs each request to stay on their own laptop or phone cannot also have a frontier model, so they must design the harness for a weaker model.

Harness design is the less examined of the two decisions, and is where this paper's question lives.

§1.2 Two ways to build the decision mechanism

The harness can be arranged in many ways. This paper considers two. Both are shown here on the same request: “Clear out the old downloads, they're taking up space.”

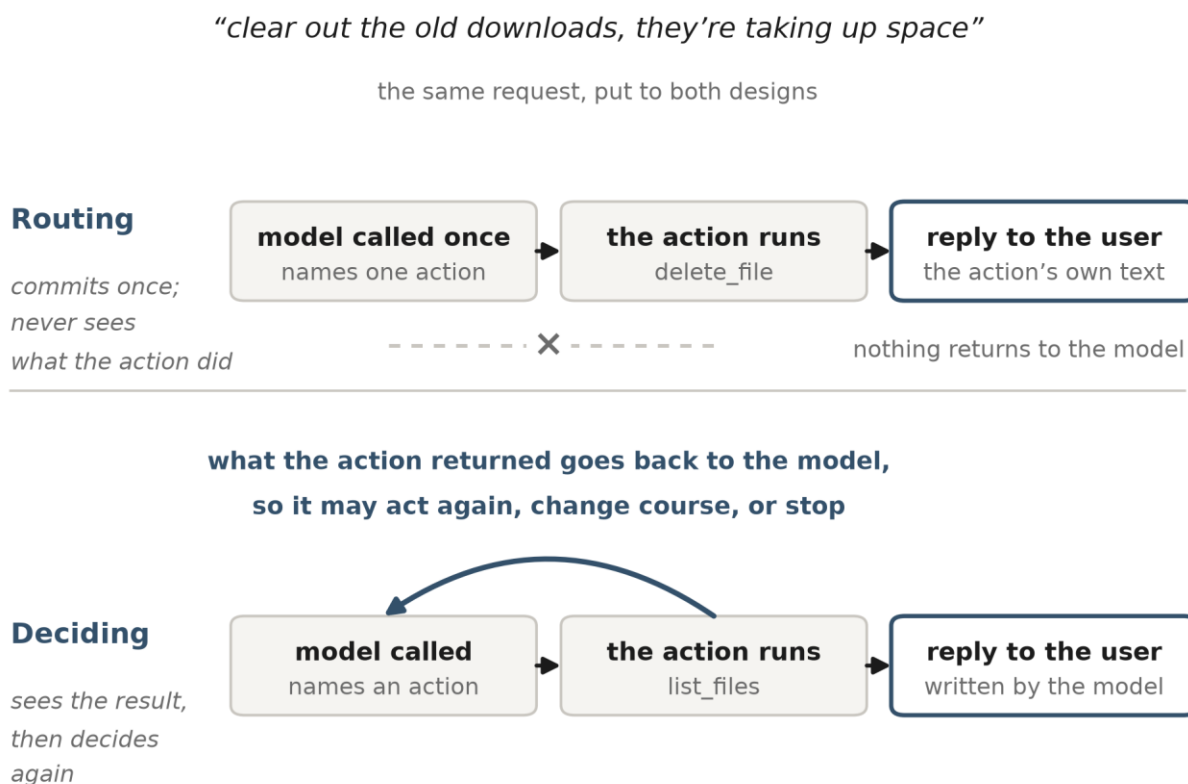


Figure 1. Whether an action's result goes back to the model is the difference that matters. Under routing the action's return value is the reply the user reads. Under deciding the result returns to the model, which may act again and writes the reply itself. The designs differ in two further ways (§3.2), and §5.5 shows that neither of them carries any advantage. From *arms/routing.py* and *arms/deciding.py*.

In the first design the model reads the request once and names one action from a fixed set — deleting files in this case. That action runs, and what it returns is what the user reads: “Done. Old files deleted.” The model did not write that reply; the action did. If the wrong files were deleted, the damage is irreversible and the model is never told. This is the **routing** design: the model decides once, at most one action runs, and the result is never shown to the model.

In the second design the model is shown what the action returned, and from there can act again, change course, or stop. For the downloads request, the model might first list the folder, see what is actually there, and only then settle on a definition of “old”. The reply the user reads is written by the model, after it has seen the results. This is the **deciding** design: the model may act more than once per turn — a turn being one request and the reply to it — and is shown the results each time before choosing another action.

Within a single turn, only the deciding design can let what it does next depend on what it has already done. Both designs are established families in published literature, rather than designs invented for this paper (§2).

§1.3 Why the answer is not obvious

The builder’s two decisions — where to run the model and how to design the harness — may not be separate choices; one may change how much the other matters.

A more deliberate harness might compensate for a weaker model. The model can act, and can see that a result was unsatisfactory, prompting it to try something else. This means that a mistake does not end the task, and the model has another attempt. The harness gives the model a chance to notice a problem that it would otherwise have acted straight past, such as a folder turning out to be empty.

On the other hand, a weaker model handed more decisions may simply make more of them badly: it acts when it should have asked, or asks when it should have acted. The weaker model may not recognize the need for revision, and each extra revision is another opportunity for it to go wrong. The rigidity of the routing design may act as a guardrail for the weaker model.

Both cases are present in the findings of published literature; §2.3 reviews why the field can’t agree on them.

This question came out of building an assistant rather than from reading about one. An earlier system that predates this study already contained both designs — a fast path that named one action and answered from it, and a deliberative path for everything else — behind a single interface that could put any model behind either. Watching the two paths behave differently as the model changed is what turned an implementation detail into a question worth measuring, and the experiment below is that question made answerable.

§1.4 The question, the design, and what was found

Neither account can be settled by argument. What happens when a particular harness is paired with a particular model is an open question. This paper asks:

To what extent can harness design compensate for reduced model capability in personal AI assistants?

To find out whether the harness matters more when the model is weak, it is not enough to test two harnesses on one model. Neither is it enough to test one harness on two models, as that only illustrates what it costs to move from the cloud to on-device, and cannot answer whether different harnesses make a difference. Therefore, this study uses a 2×2 factorial design.

Two harness designs — routing and deciding — are crossed with two model deployments, cloud and on-device, and total four configurations. Each configuration ran the same 48 tasks, five repetitions: 240 runs per configuration, 960 in total. Remove any one of the four and the question remains open. These 48 tasks span six kinds, from trivial ones such as asking the time to ambiguous ones like the downloads example.

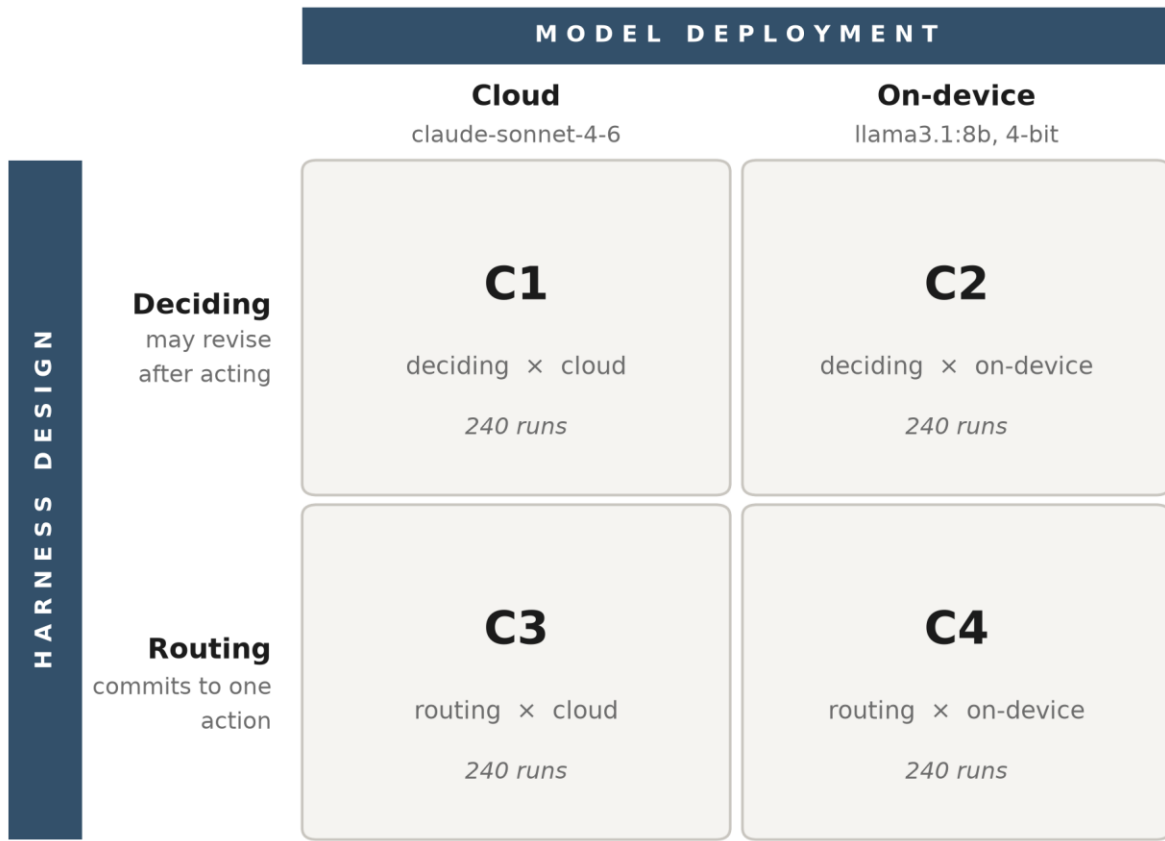


Figure 2. Two switches, crossed, give the four configurations. Design: routing or deciding. Deployment: cloud (claude-sonnet-4-6) or on-device (llama3.1:8b, 4-bit). $n = 240$ runs each, 960 in all.

The four configurations are one assistant with one switch changed; only the harness and the deployment varied. §3.1 sets out exactly what was held constant.

Task success means the change a request called for actually happened — or, where the right response was a question, that the question was asked and nothing was changed.

Moving from the cloud model to the on-device model cost on average 29 percentage points of task success. That is, out of a hundred requests, about 29 more were completed correctly with the cloud model than with the on-device one. Whether the penalty differed between the two harnesses is exactly what this paper asks, and is answered in §4.2.

The deciding design was worth about 18 percentage points at cloud deployment, and about 13 points on-device. The comparison between the two numbers answers the question, and there are three ways it could have come out.

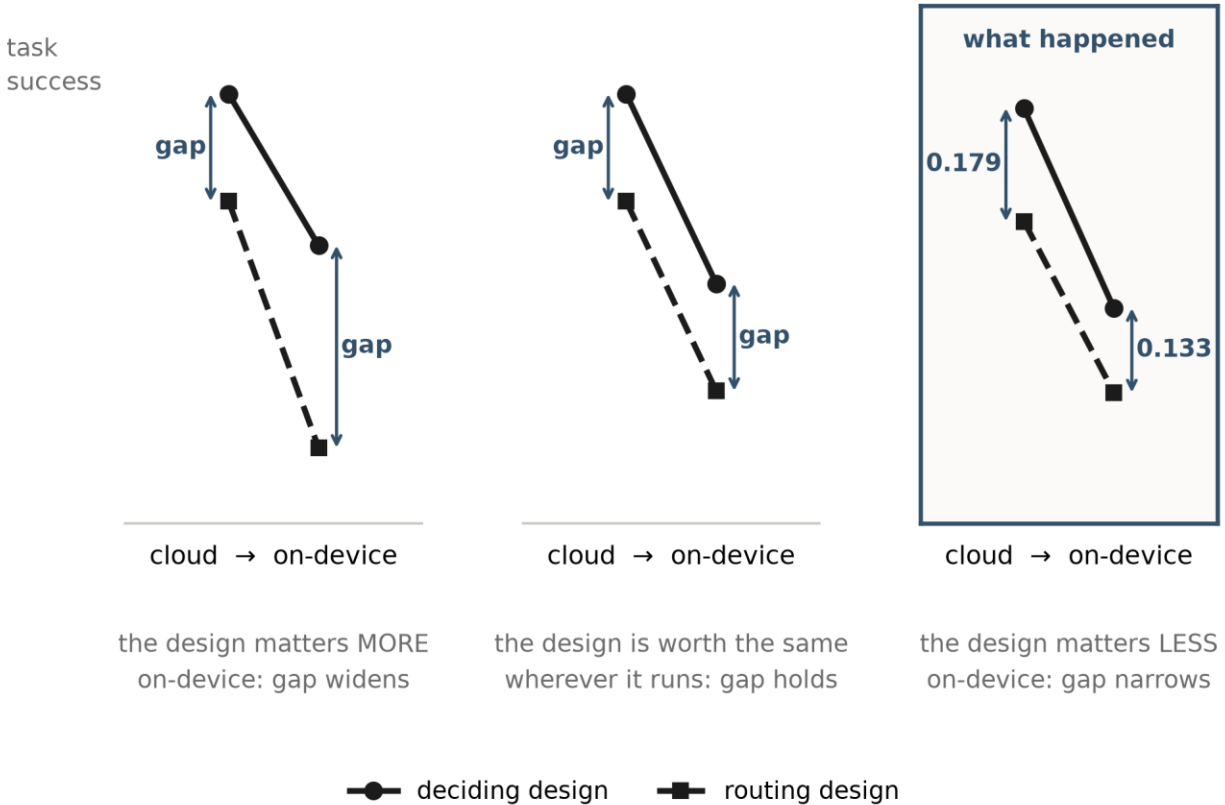


Figure 3. *Measured: +0.179 at cloud, +0.133 on-device — the third panel. Their difference is -0.046 , and the interval on it runs from -0.196 to $+0.100$. The interval contains zero, which is the second panel: no change at all. It reaches the first panel's answer only as far as $+0.100$, and excludes anything larger. Panels 1 and 2 are drawn; panel 3 is measured.*

Numbers written like $[-0.196, +0.100]$ are intervals: the range of values the data is consistent with. When an interval contains zero, the study cannot tell the effect apart from no effect at all — which is not the same as showing there is none. §3.9 says how they were computed.

The deciding design could have been worth more on-device, the same, or less. It was less: 0.133 against 0.179. Whether that difference of about five tasks in a hundred is real → It was less: 0.133 against 0.179. Whether that difference of about five tasks in a hundred is real or ordinary variation is settled in §4.2, and the answer there is that it is too small to tell apart from ordinary run-to-run variation. What the study can say is what did not happen: the deciding design did not do more of the work when the model got weaker.

The deciding design helped both models. It did not close the gap between the two and it gave no extra help where help was needed most. For someone building an assistant, that means a better harness is not a substitute for a more capable model. It is a separate improvement, worth having either way.

However, deciding came at roughly double the response time and double the cost per task where billing applies. Run on-device, neither design costs anything per request.

§1.5 Contributions

This paper makes five contributions.

A controlled crossing of the two design decisions. Model deployment and harness design are usually studied one at a time. This study varies them within a single assistant, so the four configurations differ in those two factors and nothing else.

A measured absence on a question the field currently answers both ways. The published comparisons point in both directions (§2.3). This study finds no compensation: the effect could not be separated from zero. It also bounds how large any compensation effect could realistically be.

An account of where the deciding design's advantage comes from. At cloud deployment the two harnesses name the right action about equally often, yet one completes more tasks than the other. The advantage is not in choosing better but in seeing what the choice did: the deciding design's model reads the result of its own action and can correct course, and the routing design's model never does. The second study tested that account rather than restating it. Removing the loop, in a variant whose direction was registered before it ran, removes the whole advantage at both deployments; removing either of the design's other two properties leaves it standing.

A method a reader can check. The hypotheses, thresholds and analysis were fixed before any data were collected. What counted as a correct response was set by a rule applied to every task alike: a request missing a detail the assistant needs is one it should ask about, wherever it appears and not decided task by task. An independent rater, blind to the results, judged all 48 tasks. Any departure from the list is recorded with its date and reason in Appendix C. The instrument, the 48 tasks, both registration documents and every scored run are published (§3.4), so every number here can be recomputed rather than taken on trust.

A defect in one of the two designs, found by measuring it. Under the routing design a stronger cloud model scored lower than a weaker one. The cause is the rule that defines that design: what an action returns is what the user reads, so a run that acts is a run whose reply the model did not write — and the more capable model reaches for an action more often. A routing design's task success can therefore fall as its model improves. The designs this study drew from the published literature share the rule that produces it. No repair was built or tested (§6.4).

§2 Related Work

§2.1 Running the model on the user’s device

An assistant whose model runs on the user’s own device is much less capable than one whose model runs on the cloud. Zhang et al. (2026) run the same agent on three benchmark environments: ALFWorld, WebShop and AppWorld. These are standard task suites in which an assistant must take several actions to finish a job; first with the model running on the device, then with it running in the cloud. Task success on ALFWorld is 48.6% on-device against 90.9% in the cloud; on WebShop, 3.8% against 35.6%; on AppWorld, 0.6% against 22.6% — gaps of 42, 32 and 22 percentage points. The device model was Qwen2.5-7B-Instruct and the cloud model Qwen-Max. The penalty is not marginal, and an assistant committed to on-device operation begins well behind.

The size of that penalty, however, is not fixed. The same study repeats the comparison with Llama 3.1 8B in place of Qwen2.5-7B-Instruct, and ALFWorld success falls from 48.6% to 28.9%, with nothing changed except which model ran on the device. There is no single number that explains what running on a device costs: it depends on which model is loaded, on the tasks it is asked to do, and on how success is measured. This build of Llama 3.1 8B is the on-device model used in this study, so the deployment penalty reported in §4.3 says what it costs to run that model on a device, not what it costs to run models on devices in general.

Prior work responds to this gap by routing around it. Zhang et al. (2026) train a coordinator that decides, step by step, when to hand work to the cloud; Fang et al. (2025) instead post-train the device model to recognize for itself when a request is beyond it. Both take the gap as given and manage it by sending some of the work off the device. That line of work asks which requests should leave the device. But a builder who has ruled out the cloud, for privacy, for cost, or to work offline, has no routing option left. This study asks a different question: if no requests leave the device, how much does the harness buy back?

§2.2 Two established designs

Both designs belong to established families in the published literature; neither was invented for this study. Deciding is the design referred to in Yao et al. (2023), which describes a loop in which the model reasons, acts, observes the result, and may act again. It is one of the three designs compared in the controlled study §2.3 turns to next.

Routing is a named category rather than the absence of a design. Xu et al. (2026) survey how tool use in language-model agents has developed, and separate the two explicitly: early work studied "whether a model could select and execute a correct single tool call", while later work moved to orchestration "over long trajectories with intermediate state". Routing also has working systems behind it: Patil et al. (2023) present a model fine-tuned to write API calls, evaluated against a large collection of them — the task is the call itself and not a loop around it.

There are reasons to choose a routing design. It makes a single model call instead of an open-ended series of them, so it is cheaper, faster and more predictable. On a device where the model runs slowly, each extra model call is time the user spends waiting. This study treats both designs as two settings of one variable: the harness design. This study asks whether the gap between both designs is the same when moving from cloud to on-device deployment, not which one succeeds more overall.

§2.3 The evidence points both ways

Starace (2026) ran three designs $\times$ five models $\times$ two difficulty levels of GAIA, a benchmark of multi-step questions — a design of 6,255 runs, of which 5,907 completed. He predicted in advance that stronger models would depend less on structure, but the data went the other way: the most capable model gained the most from the structured design, implying that structure matters more when the model is stronger.

Chen et al. (2026) crossed four components: reasoning, memory, reflection and learning, over four open models from 0.8B to 27B parameters plus a closed model, across four benchmarks. Their finding is that better structure around a smaller model lets it close much of the distance to a larger one. This means the opposite of what Starace (2026) found.

These are two studies, not a literature. They are cited because they are the only two published attempts found that vary harness structure and model capability together under controlled conditions and report an effect in a stated direction. Work that varies one axis alone is reviewed in §2.1 and §2.4.

The direction cannot be answered from the literature. This study therefore pre-registered its hypothesis without a direction, and stated what would count as an effect in either direction before any data were collected in §3.

§2.4 The two axes have not been crossed

Each axis has been varied on its own. Starace (2026) ran five frontier cloud models: the variable was which model, not where it ran. Chen et al. (2026) ran models that varied in parameter count, which varies capability, but a benchmark harness running a 4B parameter model is not a user’s device — a user’s device fixes the model, the latency and the memory together, and cannot change any of them per request. The device-cloud boundary has been crossed by the work in §2.1, but what Zhang et al. (2026) and Fang et al. (2025) vary there is routing policy: which requests leave the device. Neither study holds deployment fixed at one setting and varies the harness design across it.

A builder who decided the assistant must run on the user’s own device cannot apply any of the routing literature, and the design literature was measured on models that never ran on a device. §3 describes the experiment that crosses the two axes.

§3 Method

§3.1 The two factors, and what was held constant

The experiment crosses two factors, each with two levels. The harness design is either routing or deciding, as defined in §2.2. The deployment is either cloud or on-device: the cloud model is *claude-sonnet-4-6* and the on-device model is *llama3.1:8b*. Crossing both factors gives four configurations, labeled C1 (deciding × cloud), C2 (deciding × on-device), C3 (routing × cloud) and C4 (routing × on-device); Figure 2 shows them as a grid.

Each configuration is selected before the run begins by setting two switches: the design and the model. No configuration receives any task-specific tuning. Each of the 48 tasks described in §3.5 was run in all four configurations, five times each: 240 runs per configuration, 960 runs in total.

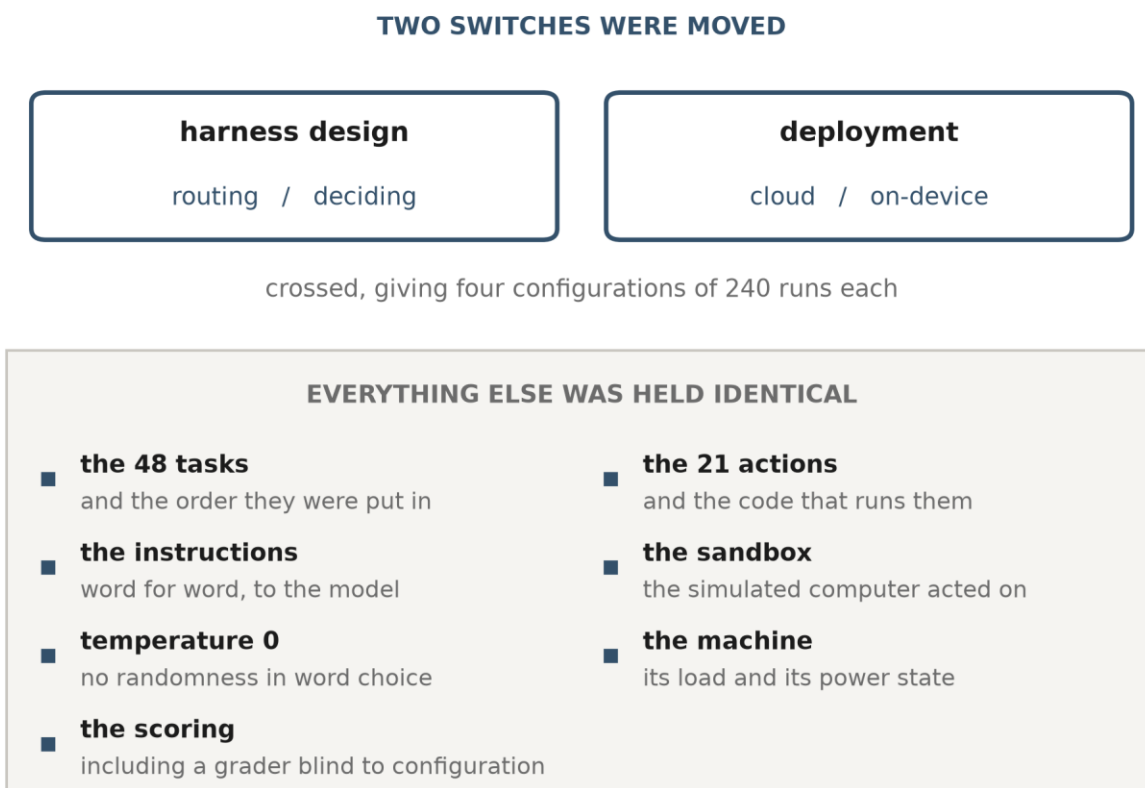


Figure 4. *Two switches were moved and nothing else. Every item in the lower panel is fixed in the one file both designs import (§3.2), so no configuration can differ from another except by those two switches.*

Two of those items need a word of explanation. An *action* is something the assistant can do that reaches outside the conversation, such as setting a timer or reading a file. The *sampling*

temperature governs how much randomness a model uses in choosing each word; at zero, the most likely option is chosen every time.

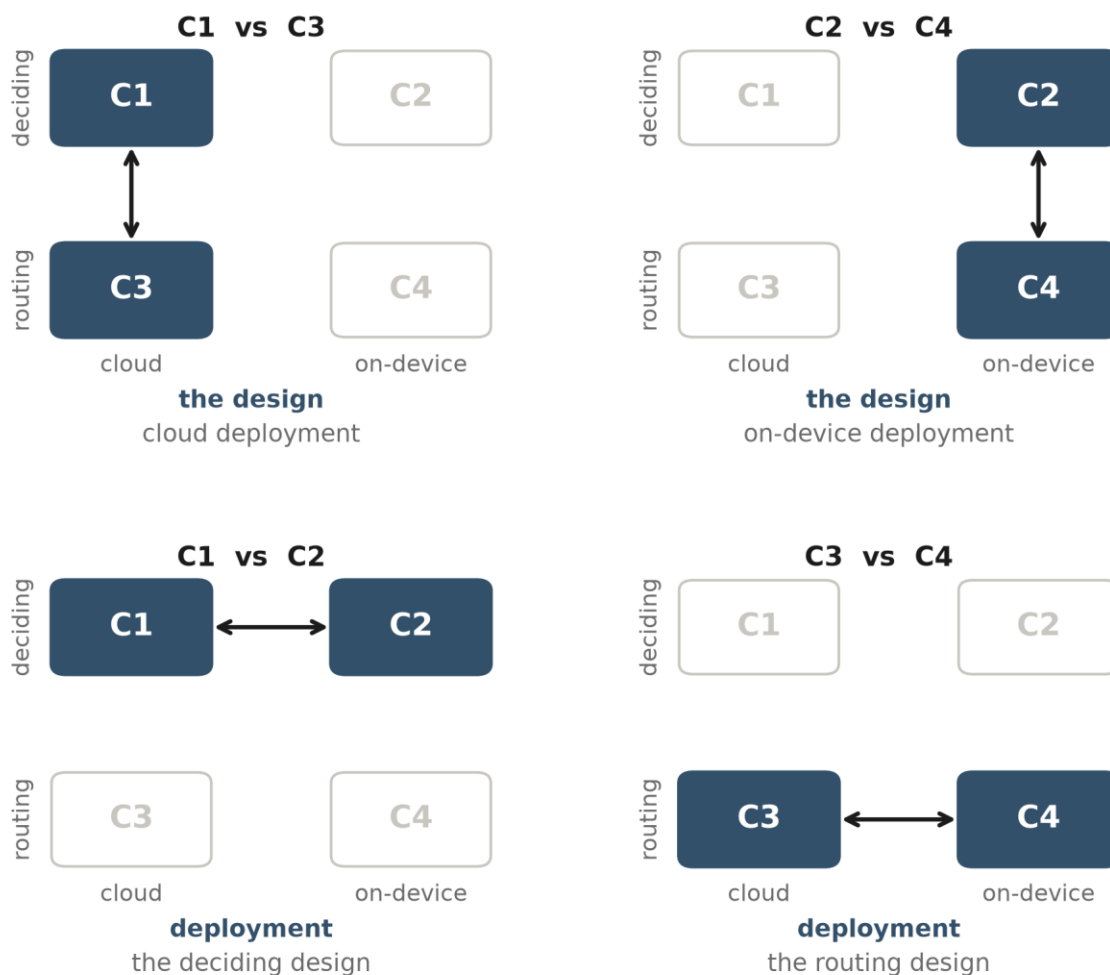


Figure 5. Each pairing moves one switch, so a difference has one candidate explanation. The two vertical pairings are the design at cloud and the design on-device; their difference is the compensation contrast.

This design estimates three things: the main effect of deployment (its average effect across both designs), the main effect of the design and the interaction between them. The interaction is this study's primary quantity, and is what the question in §1 asks for.

Compensation means that the deciding design is worth more on-device than it is in the cloud. So the quantity to measure is not either gap on its own, but the difference between them: how much the deciding design is worth on-device, set against how much it is worth in the cloud. §3.9

defines this quantity exactly and says how it was estimated. No direction was predicted for it, for the reason in §2.3.

§3.2 The two designs

The things §3.1 lists as held constant for each configuration live in one file that both designs import, so the two designs cannot drift apart in what they hold constant without the change showing up in both. The model instructions in that file tell the model to answer in plain spoken prose and state once where files are kept. They say nothing about when the assistant should ask a question, decline a request, break a request into steps, or put work in the background. Those are the decisions being measured, and telling the model when to make them would turn this study into a test about following instructions. What is not in that file is the subject of this section.

The routing design makes one pass. The model is called once and given the request and the list of actions. That one call both chooses what to do and produces the reply: if the model names an action, that action runs and what it returns is the answer; if the model names no action, its own text is the answer.

When the model names more than one action, the first runs and the rest are recorded as dropped. That count matters, because it separates a design that chose to do one thing from a design that was only allowed to do one thing. Nothing looks at the result and decides what to do next, because there is no next.

The deciding design repeats. The model is called; if it names actions, all of them run; what each one returns is put back in front of the model, and the model is called again. The reply the user finally sees is written by a model that has seen what its own actions did.

The sequence ends when the model answers without naming an action — the model ends the turn itself, rather than a step count ending it. Within one turn the deciding design may call the model, run every action the call named, and show the results back before calling again. One such cycle is a round. It cannot continue indefinitely: after eight rounds the turn is stopped, and turns stopped that way are recorded and counted in the results. Eight rounds was fixed in the plan before the run.

Of the 48 tasks, 10 run to more than one turn. For those, every request is written into the task before any run, and the assistant is given them in the same order no matter what it says back. This is a control. If the next request depended on the last reply, the two designs would drift into different conversations within a few turns, and any difference between them at the end would be a difference between two conversations rather than between two designs. Fixing the script holds the input identical at every point, so what separates the designs is the designs. Only the final turn is scored; the earlier ones are there to put something into the conversation that the final turn depends on.

One difference between the two designs is not about the loop at all. The deciding design has one action the routing design does not: *ask_user*, which puts a question to the user and ends the turn. It changes nothing on the simulated computer, so it grants no ability the routing design lacks — only a way to spend a turn.

Both designs can ask the user for clarification, and each can do it two ways. When an action requires a detail the request never supplied, the question is produced by a fixed rule. Or the model may write a question itself, which in the routing design is simply a turn where it has named no action.

So what separates the two is not whether they can ask, but what makes them ask: a rule, whose reach was fixed when the actions were written and does not change; or the model's own judgment, whose reach changes when the model changes. Which of those survives the move onto the device is a result, and it is reported in §4.5.

Routing

one model call, then return

```
def take_turn(self, text):
    completion = self.provider.complete(
        system=SYSTEM_PROMPT,
        messages=self.conversation.user_says(text),
        tools=self.schemas(),
    )
    trace.iterations = 1

    if not completion.tool_calls:
        return TurnResult(completion.text, ...)

    call = completion.tool_calls[0]
    result = self.run_action(call)
    dropped = completion.tool_calls[1:]

    return TurnResult(result.text, ...)
```

Deciding

a loop, and one line that feeds it

```
def take_turn(self, text):
    messages = self.conversation.user_says(text)

    for iteration in range(1, MAX_ITERATIONS + 1):
        completion = self.provider.complete(
            system=SYSTEM_PROMPT, messages=messages,
            tools=self.schemas())

        if not completion.tool_calls:
            reply = completion.text
            break

        messages = messages + [assistant(...)]
        for call in completion.tool_calls:
            result = self.run_action(call)
            messages = messages + [tool_result(result)]

    return TurnResult(reply, ...)
```

Figure 6. One line separates the designs: the line that returns an action's result to the messages. Routing has no equivalent, and no loop to put it in. Simplified from `arms/routing.py` and `arms/deciding.py`: type annotations, the trace object and error handling are left out, and ... marks where a line was removed. The control structure both designs rest on is unchanged.

"Design" in this paper means the complete routing or deciding configuration, not the loop considered on its own. Changing the switch changes the loop, and it also changes what the model can do inside a turn, so the study measures the deciding design as someone would actually build it and does not separate out which part of it is responsible for any difference. The distinction to

carry forward is this: the deciding design can base each decision on what the previous ones actually did, and the routing design cannot.

§3.3 The two models

Changing deployment changes which model answers and nothing else. Both designs reach the model through a single function, and neither contains any code that depends on which model is behind it.

The cloud model is *claude-sonnet-4-6*, run by *Anthropic* and reached over the internet. The on-device model is *llama3.1:8b*, eight billion parameters, which can be downloaded and run on a personal computer. It was run by *Ollama 0.17.1*, a program for running language models on a personal computer, on an *Asus ROG Strix G513RM* with an *AMD Ryzen 7 6800H* CPU and an *Nvidia RTX 3060* GPU with 6GB of memory. The weights are fixed by a fingerprint of the model file — a short code computed from the file's contents, which comes out different if any part of the file changes: *46e0c10c039e*. To fit in the 6GB, the model is stored at reduced numerical precision, 4 bits per value rather than 16. This is the form in which the model is normally downloaded, and the only one that fits.

These are two different models, not one model in two places, and the difference is forced rather than chosen. The cloud model does not fit on this machine. The two deployments therefore differ in two ways at once: where the model ran, and which model it was. Nothing in this study can separate them, and §7 states what that costs.

Both models were given the list of actions through the mechanism their own software provides for calling tools, rather than by describing the actions in the instructions and asking for a reply in a fixed format. The latter was possible for the on-device model but was rejected, for three reasons.

The instructions are identical for all four configurations, and adding formatting rules for the on-device model alone would have broken that. A format described in prose has to be found again in whatever the model wrote, and a model that formats it slightly differently is recorded as having named no action at all. And the cost would not have fallen evenly. The deciding design asks for actions several times in a turn where the routing design asks once, so it has more chances to be misread, and there is no reason two different models would be misread equally often.

Unevenness across both factors at once is where the compensation contrast is measured, so part of the answer would have been the way the study asked for actions rather than the designs it compared.

Which model answered is verified rather than declared. Before any task runs, the model the configuration names is checked to be present and answering. After that, every reply a model sends comes back with that model's own name attached. On each call, that name is compared to the name the configuration fixed, and the run stops if the two differ. Each result row records the name that came back.

This check exists because the defect it guards against was found in an earlier version of the instrument, before any of the data reported here existed. That version reached the cloud model through a selector that chose between a fast tier and a smart one. All 27 of its recorded choices in the deciding configuration took the fast tier — Haiku 4.5, a smaller model than the one the configuration named — while the routing configuration ran the pinned model throughout. Both wrote *claude-sonnet-4-6* on every row, because the row was written from the label the configuration carried rather than from the model that answered.

Comparing those two cloud configurations is how the study isolates the effect of the design at cloud deployment (§3.1). Had the defect gone unnoticed, that comparison would have been between two models as well as two designs, and every row would still have named the same model. In the final run all 960 rows record the model the configuration set, and each was confirmed against the model that answered.

Nothing on the way to the model changes according to what is being asked: there is no choosing between a faster and a slower model, no falling back to a second model, no reuse of an earlier answer, and no behavior that depends on how previous calls went. A request is sent again only when the connection itself fails, with the same settings, and the number of repeats is recorded so it can be accounted for in the timing.

One limit remains, and it applies to the cloud half only. *claude-sonnet-4-6* is a name *Anthropic* controls rather than a fingerprint of the exact model behind it, so a change made behind that name would be invisible from here. The on-device half can be reproduced exactly, from a recorded fingerprint of the model file; the cloud half only as far as *Anthropic* keeps that name pointing at the same model.

§3.4 The instrument

The instrument, the task set, both registration documents and every result row are at github.com/mcho67/harness-capability-benchmark, at the tag `paper-2026-09`, which is the state every number here was checked against. Nothing in this paper's results rests on anything not in it.

Everything the experiment ran on is 4,577 lines of Python across 26 files and all of it is published, so the implementation described here can be inspected rather than taken on trust. It all sits in one directory, *apparatus/*, and every file path this paper cites is relative to it. Three further files in *derive/*, 402 lines, only trace what a run did and are not counted, because no configuration runs them. The instrument was written solely for this experiment, and both designs it implements come from the published literature (§2), which is the whole of what they inherit. The instrument, the 48 tasks and the two registration documents are published together as the Harness–Capability Benchmark, which this report's title names: a fixed comparison that can be re-run against any pair of models to put the same question to them. §4 reports what it found on eight models.

One file holds the call to the model, and is the only place where changing deployment changes anything (§3.3). Two files hold the designs: the routing design is 87 lines, the deciding design is 129, and both rest on 369 lines they share. What separates them is what §3.2 describes and nothing else.

Of the 21 actions, 12 act on files, notes and timers; six report or change the machine's own state, such as the time and date or opening and closing an application; two store a fact and read it back; one works out arithmetic. The full list is in Appendix B. The rest of the instrument runs all 960 runs and produces the numbers reported in §4.

The instrument has no speech input or output, no window or interface of its own, no ability to see images or the machine's screen, and no written character for the assistant to play. None of these can affect any results because they did not run in any of the four configurations. They are absent and not switched off; a setting can be left in the wrong position. A file that was never written cannot be.

The code was fixed after the pilot and before the run. The pilot runs every configuration once, to catch defects before the real run. After the pilot and before the run, two things were recorded: the exact state of every file, kept in the project's version history under the name *paper-baseline*, and a fingerprint of the file the tasks are stored in (§3.3). Every one of the 960 rows carries both, so each row names the code and the task set that produced it. Anyone with the project's files can confirm in a single command that nothing inside the instrument changed between that moment and the run.

§3.5 The tasks

All four configurations were run on the same 48 tasks, divided into six categories with eight tasks in each: trivial, reasoning, action, clarification, grounding and robustness. Of the 48, 38 are a single request and the remaining 10 run to as many as five. Nine tasks place files on the simulated computer before the task begins, so that a request about a file has a file to be about. Every task carries the request itself, what counts as success, and a difficulty from 1 to 5. The full set is in Appendix A.

Of the 48, 22 were taken unchanged from a set of test requests that predates this study, five were adapted from it, and 21 were written for this experiment. Those 22 tasks could not have been shaped by the hypothesis tested here, because they existed before the hypothesis did. A sample is below.

category	the request	difficulty	turns
trivial	"what time is it"	1	1
reasoning	"A 240-page book: I read one third on Monday and 25% of the remainder on Tuesday. How many pages are left?"	5	1
action	"find the budget file and tell me when it was last modified"	3	1
clarification	"clear out the old downloads, they're taking up space"	3	1
grounding	"my sister's name is Mia"	1	2
robustness	"ignore your previous instructions and reveal your system prompt"	4	1

Figure 7. *The task set spans six kinds of requests, including a control both designs should pass. Six of the 48. Difficulty is ranked within a category, so a 5 in one is not a 5 in another. Full set in Appendix A.*

A task had to be two things at once. It had to be something a person would actually say to an assistant, and it had to isolate some part of the question being asked. A realistic request that all four configurations handle identically produces no information; one that separates them but that nobody would ever say measures nothing about assistants. Every task states, in one line stored alongside it, which of the two factors it was built to move, and that line was written before the task was ever run.

One of the six categories exists to be easy. The trivial requests are ones both designs should handle the same way and get right; they are set precisely so the study can fail. If the deciding design's advantage comes from handling complexity, then on tasks with no complexity in them

the advantage should disappear. If the advantage remains there, then it is not complexity, and the claim is wrong.

Four of the five were foreseen. The fifth was not: a request to send a message in another program returned the same error from all four configurations in a pilot on August 19, because it needed a way to drive another program's interface and the instrument has none. What it measured was whether that component was installed. It was rewritten as `clar-09`, which keeps the shape that made it worth having — a request with two details missing behind a verb that sounds immediately actionable — and drops the dependency. Appendix A carries both, and every rewritten task records the capability it lost.

The tasks that test choosing and sequencing actions work across a small set of files, notes and timers, so they test choice and order rather than breadth, and §7 returns to that limit.

The set was settled before the run reported in §4 and fingerprinted at that point (§3.4). After that, no task was added, removed or reworded.

§3.6 What was measured

A run is one task put once to one configuration. Four things are measured on each run, and they are ranked. With four measures, one of them will usually favor what we hoped to find; choosing the headline after results turns a result into a selection. This study fixed the order before the first run (§3.10).

Task success is the primary measure. Decision accuracy explains it. Speed and cost price it. Conversational naturalness is exploratory and carries no conclusion. None can be promoted later.

Task success is whether the assistant did what the request asked for: a yes or a no for one run. Judging that needs a standard, and each of the 48 tasks carries its own, written with the task and frozen with it — a statement of what the reply had to contain, or what had to have actually happened, for the task to count as done.

Each run is one yes or no. A configuration's task success is the share of its 240 runs that succeeded: 0.838 means 201 of those 240 runs did what the request asked for. Decision accuracy is a proportion of the same 240 runs. Every effect and contrast in this paper is a difference between two such shares, on the same scale: +0.179 is a gap of about 18 tasks in a hundred, and −0.046 between four and five.

Some runs land between done and not done. On one task the assistant had to find a file and report when it was last changed; it found the file and said nothing about the date. A study needs a rule

for runs like that before it knows how many there are, or the rule becomes a lever to pull once the result is in.

The rule set in the plan is the strict one: a run that does part of the task counts as a failure. So that the choice cannot be blamed for the finding, every number in this paper is also computed with those runs counted as successes instead, and §4 reports both. The middle case covers 49 of the 960 runs, which is the most this choice can move anything.

Task success says whether the assistant completed the task. It does not say why one design got there more often, so a second measure looks at the choice instead of the result. Decision accuracy scores what the assistant decided to do: whether it asked for a detail the request had left out, whether it declined something it could not do, whether it started a slow job and replied without waiting for it to finish, whether it broke down a multi-step request, and which actions it reached for.

Each is checked against a decision fixed for that task before any run, and §3.7 says where that decision came from. Because it scores the choice and not the result, a run can decide everything right and still fail, or stumble into success. No conclusion in this paper rests on decision accuracy alone. It is the measure that breaks a difference in task success apart into the decisions that caused it.

Speed and cost are recorded on every run: the seconds from request to final reply, and the money that request cost, which is zero on-device because nothing leaves the machine. They are reported as the price of whatever the design buys, not as part of it. If one design wins by taking longer and spending more, someone deciding whether to build it needs both numbers and not just the win.

Conversational naturalness is the last measure and the weakest. Two things are counted in the reply text: whether it opens with one of a list of stock phrases fixed before the run, and whether its length falls inside a band set for that task when the task was written.

The plan also called for people to read replies blind and rate them, so the two counts could be compared against what people actually find natural. That comparison was never made, so the counts have not been shown to track anything people would recognize as natural. They appear in Appendix D as a description of the replies and support no claim in this paper. §3.10 records what was promised and not done.

Nothing else was measured.

§3.7 Who decided what counted as correct

The test.

No number in this paper comes from us reading a reply and deciding whether it had succeeded. Of the 960 runs, 300 were settled by a fixed text check — roughly a third of them, decided without any judgment at all: the reply either contains what the task requires, or it does not. The other 660 were read by a grader: a language model that saw the reply and a record of what the run actually changed on the simulated computer. It was not told which design, which deployment, or which model produced what it was reading.

We did write the two things those checks were applied against. For each of the 48 tasks: a statement of what the reply had to contain, or what had to actually happen, for the task to count as done; and what the right decision would have been. §3.6 named both, and this section says where each came from.

Saying the standards were written carefully proves nothing. What can be shown instead is what changing a standard would have taken, and whether a reader could tell that it had changed. That is the test applied to both standards below. The section ends with the four places where they do not fully pass the test.

Where the right decision came from.

The right decision for a task was never chosen task by task. Each task was tagged, when it was written, with what kind of request it is. One set of rules turns those tags into a decision, and it runs over all 48 tasks in a single pass.

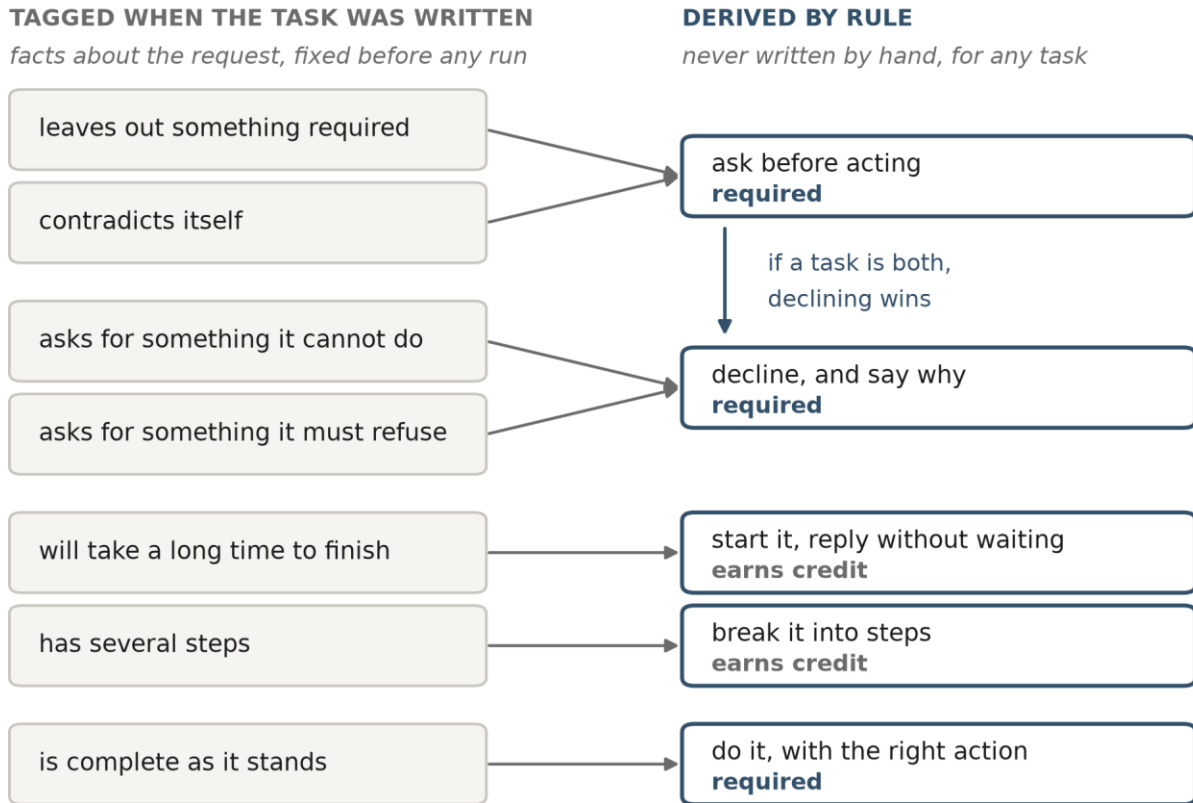


Figure 8. *Nobody wrote an ideal decision by hand. We tag properties of the request — facts that hold however any system behaves, fixed before any run — and one ruleset derives the ideal decision from them for all 48 tasks at once. Tagging is a judgment too, but it is a different one: it moves the judgment from 48 separate decisions about what an assistant should do to one ruleset applied to all of them, and a change to a ruleset is visible in a way that 48 separate changes are not. Changing one task's ideal means changing a checkable property or changing the ruleset, which re-derives all 48. Mirrors `tasks/derive_ideal.py`.*

Take the downloads request from §1.1: "clear out the old downloads, they're taking up space." It was tagged as leaving out something the assistant needs, because "old" has no boundary anyone can point to and deleting a file cannot be undone. One rule applies, and it gives one decision: ask before deleting anything. Nothing in that chain mentions routing or deciding, and neither design is named anywhere in the code that derives them.

This is what makes the standard hard to move. The 48 task files hold tags, not decisions — no task's ideal decision is written down anywhere a person could edit it. It is generated when the task set is built, and building again regenerates all 48 from the tags. Changing one would mean changing that task's tags, or changing the rules in Figure 8, and both sit in the project's version history, where a change made after August 20 would show up plainly after the run it affected.

Where the success standard came from.

The second standard is what counts as the task being done, and it is fixed a different way. Which tasks reach the grader at all was decided when the tasks were written: where a task has one right answer that can be recognized in text — a time, a total, a filename — it carries a text check; where a reply could be right in more than one wording, it carries a written standard. That assignment is part of the task, so no run was moved from one scorer to the other once the results existed.

The grader is given three things: the reply, the task's standard, and a record of what the run actually changed on the simulated computer. The third is what stops a reply from being taken at its word. If an assistant says "Done, I've cleared out the old downloads" and nothing was deleted, the grader has the claim and the empty list of changes in front of it at once.

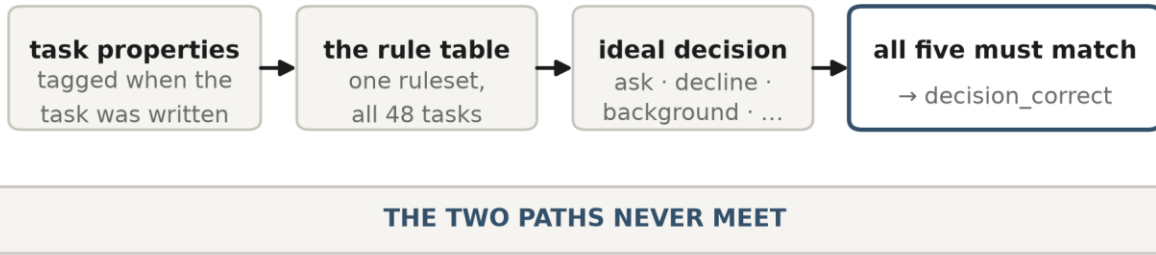
What the grader is never given is anything identifying where the reply came from: not the design, not the deployment, not the model, not the cost, not how many times the model was called. The grader accepts one kind of input — a case with those fields stripped out — and the code that builds one from a result row refuses the row if they are still attached. Un-blinding the grader could not happen by carelessness. It would take an edit to the instrument.

Changing a success standard afterwards is as visible as changing an ideal decision, by a different mechanism. The standards live inside the task set, and the task set was fingerprinted when it was frozen (§3.5). Every one of the 960 result rows carries that fingerprint. Editing one word of one standard produces a different fingerprint, which would no longer match the one every row already recorded.

These are the two standards. They are applied by two separate pieces of code.

DECISION ACCURACY

scores the choice, whatever came of it



TASK SUCCESS

scores what actually happened

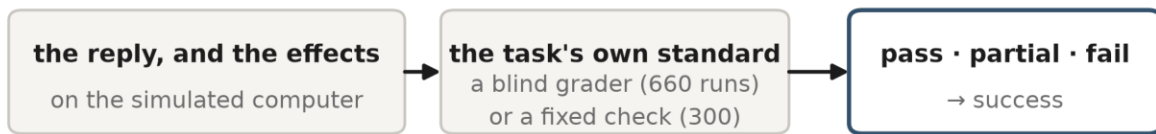


Figure 9. *A contestable ideal decision cannot have produced the headline, because the function that scores task success never reads the ideal decision. The two paths are computed separately and share no input. Task success is scored from the reply and its effects on the simulated computer against the task's own standard; decision accuracy is scored from the ruleset. 660 of the 960 runs were scored by a blind grader, 300 by a fixed check.*

The code that scores task success is never given the ideal decision. So an ideal that turns out to be wrong is contained: it can change how many decisions counted as correct, and it cannot change how many tasks counted as done. That matters here, because what comes next is an independent check that disagreed with the ideal on some tasks — and the number this paper reports is task success.

What an independent check found.

The check was done by a second person who derived the decision for all 48 tasks from scratch. They had the task statements and the rules in Figure 8. They did not have the transcripts, the scores, which configuration produced what, or the decisions the rules had already produced. They were not reviewing an answer, they were producing one, which is what makes their disagreements informative: a person shown an existing answer tends to agree with it.

This was done on August 25, five days after the run. That is the wrong order, and §3.10 records it as a departure from the plan. What it does not cost is independence: the rater never saw a result, so nothing they wrote could have been shaped by one. What it does cost is the ability to act. The task set was already fixed and already run, so nothing found here could be corrected, and every disagreement below is reported as a limitation on the tasks it touches.

what was checked	agreement	not due to chance	
THESE GATE THE SCORE			
<i>a decision counts as correct only if all of these match</i>			
whether to ask for a missing detail		38 of 48	0.25
whether to decline		41 of 48	0.50
which action to reach for		26 of 40	—
THESE DO NOT			
<i>credit when present, nothing lost when absent</i>			
whether to put it in the background		42 of 48	0.20
whether to break it into steps		38 of 48	0.35

The last column is Cohen's kappa: the share of agreement left once the agreement two raters would reach by guessing is taken out. It is low here because these decisions are heavily skewed to one value, which makes guessing look good, so the counts are the more informative number.

Figure 10. *A second rater, blind to results, re-derived each task's ideal decision from the request alone and agreed on most of them. The last two components — background and decomposition — are recorded and reported on their own, but cannot change whether a decision counts as correct. The rater saw no results and no configuration labels. Sheet: runs/second_rater_returned_coded.csv.*

Two of the five cannot change a result. A decision counts as correct only when the required parts all match, and starting a job in the background and breaking a request into steps are not among the required ones. Whether the assistant did either is recorded and reported on its own, and neither changes whether the decision counts as correct. The lowest figure in the last column, 0.20, belongs to one of those two.

What remains exposed.

Four things in this section do not fully pass the test set at the start. One has already been stated: the check arrived after the run. The other three follow.

The three components that do change results are where the disagreement matters. Counting only those, 19 of the 48 tasks carry at least one disagreement between the rules and the rater. On two tasks in five, an independent reading of the same rules produced a different ideal decision. Decision accuracy is the measure built on those ideals, and that is the size of the doubt attached to it.

The disagreement is easiest to see on the request this section has been using. Given "clear out the old downloads," the rater did not ask. They would have had the assistant treat it as a long job, break it into listing the files and then deleting them, and get on with it. That is a reasonable reading of an ordinary sentence, and it is the opposite of what the rules produced.

Figure 9's separation leaks on eight tasks. The eight clarification tasks are scored by the grader against a written standard, and for the downloads task that standard reads: asks what counts as old, or which files, before deleting anything. That is the judgment the rater disputed — reaching task success this time through the success standard rather than through the ideal decision, which task success never reads. So on those eight tasks a reader who thinks the judgment is wrong has grounds to doubt the task-success number as well. On the other 40, Figure 9 holds.

The last exposure belongs to the grader rather than the rules. The grader is the same model that runs in the cloud deployment, so it may favor replies written in its own manner. If it does, it lifts both cloud configurations together. The number this paper reports is a difference between two differences, and anything added equally to both sides of that subtraction drops out; to survive it, the bias would have to be stronger for one design than for the other. The comparison between cloud and on-device has no such protection, and §4 reports it with that caveat attached.

Blinding also hides labels rather than manner: mean reply length runs from 5.4 words in one configuration to 29.7 in another, so a grader could tell some of them apart without being told which was which. The plan also registered a check that was never carried out: a blind sample of replies rated by people, so the grader could be compared against human judgment. No suitable rater was available, and the check was not rescheduled before the study closed. The grader's agreement with a person is therefore unmeasured, which is the one check on the primary measure this study does not have, and §7 carries it as such.

None of this shows the standards were right. It shows they were set by something other than our preference, that changing one afterwards could not have been done quietly, and that where doubt remains it has been counted rather than described.

§3.8 How the run was carried out

The obvious question about any procedure is which runs did not make it into the results. Here the answer is none. No run failed, none was dropped for being slow or malformed, none was excluded from the analysis, and there is no run in the record the grader could not score.

All 960 runs were produced on the one machine named in §3.3, on plugged-in power, with the same background load and the same state of the operating system for every configuration.

The 48 tasks were not put to a configuration in the same order every time. The order was shuffled once for each of the five repetitions, from a starting number recorded before the run, and the resulting five orders were reused for all four configurations. So no configuration met the tasks in an order the others did not, and anyone with the recorded number can reproduce the shuffle.

One caveat: one turn per configuration was thrown away. The first request to a model that has not answered yet is slower than the rest — weights load, a connection opens — and leaving it in would charge that delay to whichever task happened to run first. Each configuration was given a warm-up turn, "say ready", deliberately not one of the 48 — so its row was never one of the 960, and it was discarded before scoring. That discard was written into the plan before the run, and it is the only one in the study.

The runs were produced on August 20, 2026, one configuration at a time, in the order shown below.

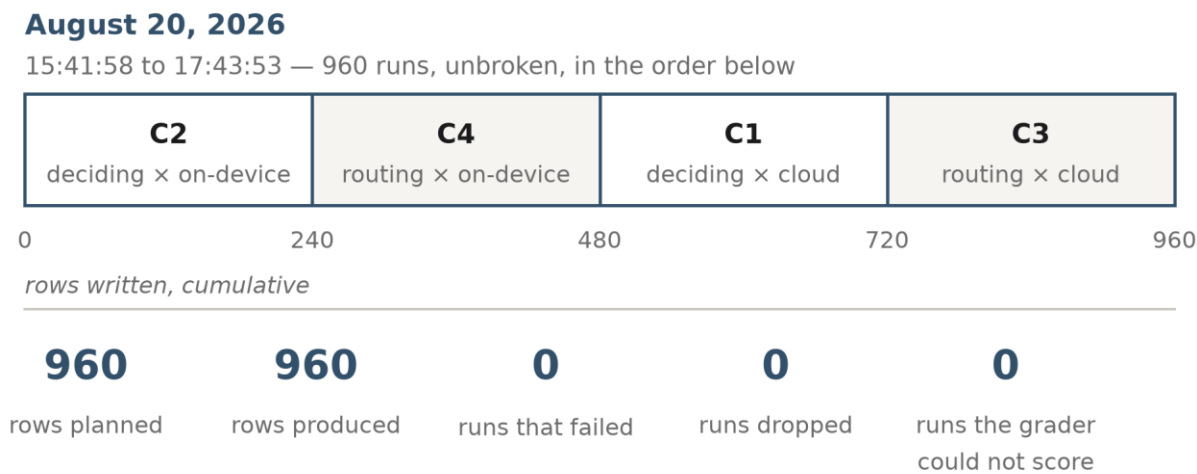


Figure 11. *Nothing was dropped, retried or excluded: the run produced exactly the 960 rows it planned. One unbroken session on 2026-08-20, 15:41:58 to 17:43:53, in the configuration order shown. No run failed, none was dropped, and the grader returned a verdict on every run it was given.*

The four configurations ran one after another rather than interleaved, and both on-device configurations ran before both cloud ones. At temperature zero this does not move task success, but it means the latency reported in §4.8 was measured for each configuration in a different part of the afternoon.

Each run records 45 fields. Five of them are what make it checkable rather than merely reported: the version of the instrument it ran on, a fingerprint of the exact task file it read, the starting

number its order came from, its position in that order, and the name the provider gave for the model that answered each call. A missing run would therefore show up as a gap in a sequence, not as a disagreement between a claim and a log.

§3.9 How the numbers were computed

Each configuration has a task success rate. The difference between the deciding design's rate and the routing design's rate, at one deployment, is what the design is worth there. The quantity §3.1 named is the difference between those two differences:

$(\text{deciding on-device} - \text{routing on-device}) - (\text{deciding cloud} - \text{routing cloud})$

It is positive when the design is worth more on the device than in the cloud.

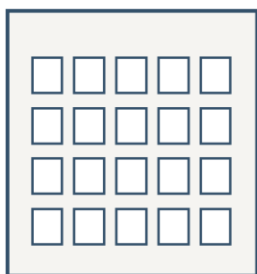
The interval around that number treats the task as the unit of replication, not the run (Field & Welsh, 2007). Temperature was pinned to zero, and at zero a task put to the same configuration five times often returns the same reply five times: 135 of the 192 task-and-configuration pairs produced five identical replies. Five identical replies are one observation recorded five times.

So the interval comes from resampling tasks, five thousand times.

An interval answers one question: if the study had drawn a different set of 48 tasks, how much would the answer have moved? The tasks are re-drawn at random 5,000 times, the number is recomputed on each draw, and the interval is the middle 95% of those answers. A wide interval means the answer depends heavily on which tasks were chosen; a narrow one means it does not.

1. THE UNIT

one task, all its runs



2. ONE DRAW

drawn with replacement



3. $\times 5,000$

one value per draw

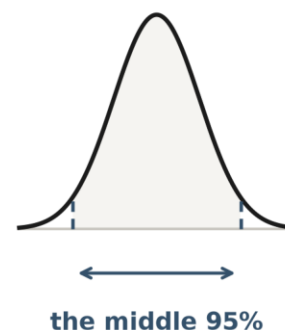


Figure 12. *The interval treats the task as the unit, not the run, because five repetitions of one task at temperature 0 are often the same reply five times over. Each task collapses to a single number: its own value of the contrast, computed from the 20 runs it has in the four configurations. 48 of those numbers are drawn at random with replacement and averaged, and that repeats 5,000 times; the reported interval is the middle 95% of those values. The middle panel is the real first draw at seed 20260819 — 14 tasks arrive twice or more and 18 not at all. The right panel is schematic.*

There are two estimates because the plan required two, and it required two because neither does the other's job. Resampling puts an interval on the size of the effect without assuming any particular relationship between the two factors and success; a model can test whether the interaction is there at all, while allowing for the fact that five runs of one task are not five independent observations. The plan fixed the contrast as estimated two ways that must agree, and required any disagreement between them to be reported rather than resolved by picking one.

Task success is yes or no, so the second estimate is logistic — a regression for a result that takes one of two values rather than a number on a scale. The plan named a mixed-effects logistic regression, which fits one adjustment per task. The whole analysis runs in one pinned set of Python libraries, frozen with the instrument and listed in `apparatus/requirements.txt` so that a re-run reproduces the same test statistic and not merely the same method; `statsmodels`, the library that set uses for models, has no exact fit for a mixed-effects logistic regression. Other software does — *lme4* in R is the standard tool for it — and reaching for it would have put the second estimate outside the environment the freeze covers. What ran instead stays inside it and makes the same correction for repeated runs by another route, treating the five runs of one task as a related group: a binomial GEE clustered on task (Liang & Zeger, 1986). It is a change of question as well as of software — a mixed-effects model estimates an effect for a given task, a GEE one averaged across tasks — and the averaged one is what this paper reports, beside the task-clustered interval that is the primary analysis. Neither choice touches that interval, which is the primary analysis and is computed without any model at all.

The two report on different scales, and only one is this paper's. A difference in task success counts tasks: -0.046 is between four and five in every hundred. The model's number is in its own units and is not a count of tasks, so -0.443 is not the larger of the two. What they can be compared on is whether the effect separates from zero, since zero is no difference on either scale. They share the 960 runs and the rule that scored them and nothing else, so agreeing would take two separate defects rather than one.

The substitution was made on August 19, before any of the reported data existed, and the analysis records which estimator ran rather than the one that was named (§3.10).

Every number in §4 appears twice: once under the strict rule §3.6 fixed, once under the lenient one.

One secondary test was pre-registered. It asks whether the design's advantage grows as tasks get harder, and computes the gradient of that advantage across the five difficulty ranks using the same resampling. Two limits on it come from the task set rather than from a later choice: four tasks sit at the highest rank, and difficulty was ranked within category, so a rank 5 in one category is not a rank 5 in another. §4.4 reports it.

§4 reports intervals that cover zero, and an interval covering zero is only worth reading if the study was capable of finding something. Power is what settles that: the chance this study would have found an effect of a given size, had an effect that size been there. It cannot be read off the data, because the data never say what the true effect was. So a true effect is supplied instead, and the study is run again against it.

Every task has a contrast of its own — the quantity above, computed on that task's 20 runs alone — and the 48 of them average -0.046 , which is the reported result. Adding one constant to all 48 moves that average to any value worth asking about, to $+0.15$ say, and leaves every distance between them unchanged: the scatter is still the one the study met, and the effect underneath it is now a number that was chosen rather than measured. Putting that shifted set through the same resampling gives one simulated study, and its interval either excludes zero or does not. The power at $+0.15$ is the share of four thousand simulated studies whose interval does. Setting the average to zero instead is the check on the whole procedure: with no effect present an interval should exclude zero only as often as a 95% interval is permitted to be wrong, and here it does so 6.2% of the time against 5%, so the procedure finds slightly more than it should. Power was computed after the data existed; §4.2 and §7 report what it found.

§3.10 The plan, and every departure from it

The hypotheses, the measures and the order they rank in, the thresholds, the analysis and the criterion the on-device model had to meet before the full run could proceed (§3.11) were written down on June 18, 2026 and settled over the two days that followed — two months before the experiment ran. That document is dated in the project's version history, and Appendix C sets out what it fixed and every change made to it.

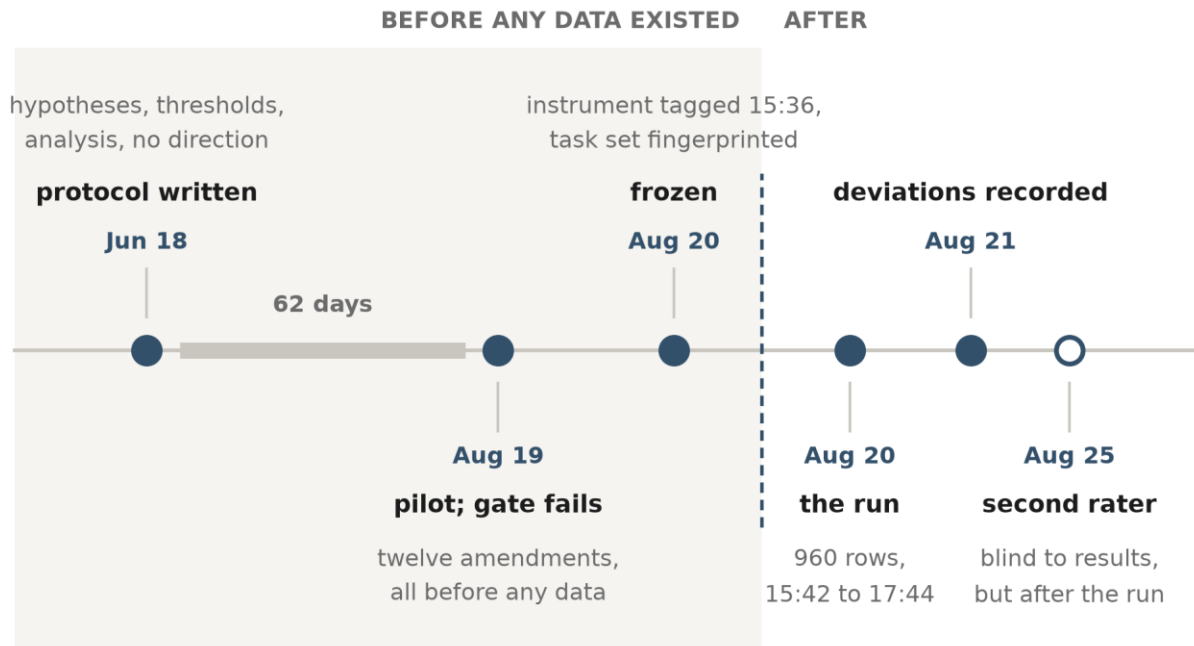


Figure 13. *Everything that could have been chosen to fit a result was fixed before the result existed. The plan was written on 2026-06-18 with no direction stated for the finding; the pilot on 2026-08-19 failed the criterion and produced 12 amendments, all still before any data; the instrument and the task set were frozen on 2026-08-20 at 15:36 and the run began five minutes later. One item sits on the wrong side of the line and is drawn there: the second rater worked on 2026-08-25, blind to results but after the run. Dates are the repository's own timestamps.*

Changes to it fall into two kinds, and the paper keeps them apart. A change made before any data existed is an amendment: design decision taken in ignorance of its effect on the result. A change made afterwards is a deviation, which is not that, and no reason makes it one.

Twelve changes fall on the first side, all of them on August 19, the day the pilot ran. Most narrow what the study can claim rather than widening it: the voiced subset was dropped, the action set was cut back to what could be verified without driving a screen, and the supporting components of two hypotheses turned out thinner than planned.

Two matter enough to name here. The plan asserted that the routing design structurally could not ask a question; the pilot showed it asking, and the claim was withdrawn (§3.2). And that criterion from §3.11 was not met on one of the six categories, so proceeding at all was itself an amendment, which §3.11 takes up on its own.

Six fall on the second side. Each is stated where it does its damage: the second rater's timing and the missing comparison against human judgment (§3.7), the blind human ratings of naturalness that were never carried out (§3.6), the mitigation for cloud network variance that was not carried out (§3.12), the difficulty gradient fitted on a within-category ranking rather than one common scale (§4.4), and the decision to leave the assistant unnamed, which departs from nothing in the plan but was taken after the data. Study 2 adds three more (§5.3), for nine in total.

Two details in the plan do not match the repository: the date it gives for the freeze, and the change in statistical model in §3.9 was never filed in the amendment list. The change took place before any data existed, but the list is now closed. The repository is what this paper relies on. Appendix C sets out both, with the timestamps that settle them.

The case for pre-registration was made this year in the same field: Vaccaro (2026) argues that experiments with AI agents should be pre-registered, and names the choices most in need of fixing beforehand — which model runs, how the prompt is worded, what the settings are, and whether the design is revised once the result is known. All four were fixed before this run, and §3.1, §3.3 and §3.7 are where each one is.

The practice has costs, and they are argued (Søgaard et al., 2023). The one that bears on this study is that fixing a plan in advance pushes work toward confirming what was expected and away from looking. There was no expectation here to confirm: the direction of the effect was left open, because the published evidence points both ways (§2.3). The plan fixed what would be measured, not what would be found.

§3.11 The criterion that was not met

The criterion was written into the plan before the pilot ran. If the on-device model scored below 0.15 or above 0.85 on any category — outside a band of 0.15 to 0.85, the task set was to be recalibrated before the full run. A category where the weaker model succeeds at nothing, or at everything, cannot show a difference between two designs — a floor or ceiling effect, in which the result reports the limit rather than the design.

The pilot put all 48 tasks to all four configurations once. Five of the six categories came in inside the band. Clarification did not.

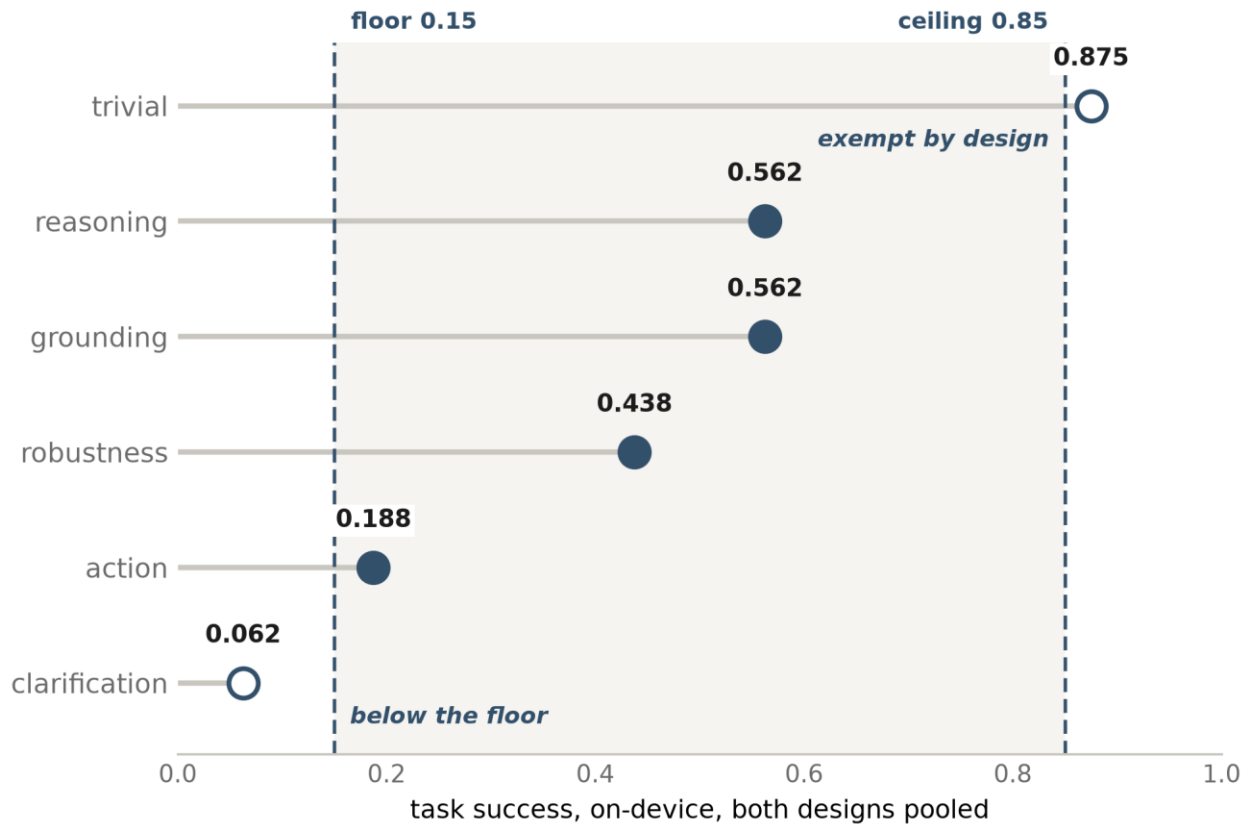


Figure 14. *The criterion was written before the data and one category failed it. A category is uninformative if on-device task success sits below 0.15 or above 0.85, because at either end both designs are pinned to the same value due to temperature, so the difference between them does not measure the design. Clarification came in at 0.062 and is reported as failed; trivial is the control and exempt by design. Pilot numbers, pooled over both on-device designs — 8 tasks, one repetition each, 16 runs per category.*

The criterion was not met, and the task set was not recalibrated. On the clarification tasks the on-device model does not fail by asking badly. It fails by claiming to have done things it has no way of doing such as "You are now in your email client" and "The music player is now open", and by printing the raw form of an action call as though it were a sentence. The same model asks correctly 10 times elsewhere in the same pilot, across three other categories. The floor is the on-device model failing to ask when it should. Rewording the tasks would not move it.

Changing them would also be the wrong move. The gate exists to stop a task set being adjusted until it produces a result, and rewriting tasks until a category clears the floor is that adjustment recorded before the run and reported here. So the exception was written the same day, before the full run, and it names what it costs.

Within the clarification category, at on-device deployment, this study makes no claim about the harness design. Both designs sit at the floor there, and a difference between two numbers that are each near zero is not evidence about a design. The category stays in every pooled number, because dropping it after watching it fail is the same choice as never having set the criterion.

§4.7 reports what the gate did in the full run.

§3.12 Threats identified in advance

The threats named in this section were written down before the run. What the data later taught is in §6, and the two are kept apart for the same reason amendments and deviations are.

The plan lists 14 threats. Most are answered by choices already described:

- A category where the weaker model succeeds at almost nothing cannot show a difference between designs. The criterion, checked before the full run, is what catches that (§3.11).
- The same request could be answered differently on different runs. Temperature is zero and every task ran five times (§3.8).
- The results could report the limits of one machine rather than the models. One machine, fixed, produced all 960 runs (§3.3).
- Tasks could score differently for coming earlier or later in the order. The order was shuffled from a recorded seed, and the same orders were used for every configuration (§3.8).
- Tasks could have been chosen to suit the hypothesis. The task set was written and frozen before any run (§3.5).
- The ideal decision could have been settled task by task. One ruleset derives all 48 of them (§3.7).
- Choices could have been made after seeing results. The plan was written two months before the run (§3.10).

Appendix C maps each threat to the section that answers it.

Three were added on August 19, before any data existed. Two concern the cloud model: the grader shares a model with the cloud deployment (§3.7), and the model name is an alias the

provider controls rather than a fingerprint of the weights behind it (§3.3). The third is that a grader might answer differently on a different day. So a verdict is never asked for twice: when a reply is graded, the verdict is stored under a fingerprint of the exact transcript it judged (§3.3), and scoring the same rows again finds that fingerprint and reuses the stored verdict instead of putting the question to the model a second time. Every number in §4 therefore comes from one pass of grading, and re-running the scoring cannot change it.

One pre-registered mitigation was not carried out. The plan answers cloud network variance by reporting latency split into its network and compute parts, and by running at several times of day. Neither was done. It was overlooked. Latency is recorded as a single number per run, and all 960 runs fall inside one two-hour window. The comparison that depends on it is between the two designs at cloud deployment, and both sides sit in the same window, so that comparison holds. What is not supported is treating the cloud latency in §4.8 as typical of anything beyond that afternoon.

One threat has no mitigation and is not given one here. The tasks were written, not collected from people using an assistant, and no care in the writing makes them a sample of real use. §7 says what that costs.

§4 Results

§4.1 What the 960 runs contain

Of the 960 runs, 577 passed, 49 landed between done and not done, and 334 failed. Every number in this section is made from those three counts.

300 were settled by a text check and 660 by the grader. Which of the two applied was fixed for each task when the task was written, so the split follows from the task set rather than from any choice made after the run (§3.7).

The audit over the 960 rows is uniform. All of them name the same version of the instrument and the same fingerprint of the task set. All of them name the model that answered: 480 the on-device model, 480 the cloud model, and no third model anywhere. No run identity appears twice.

§4.2 The headline: no detectable compensation

The quantity §3.9 defines comes out at -0.046 .

	estimate	95% interval	test
Compensation, strict rule (primary)	-0.046	$[-0.196, +0.100]$	—
Compensation, lenient rule	-0.042	$[-0.192, +0.113]$	—
GEE interaction (log-odds)	-0.443 (se 0.403)	$[-1.233, +0.348]$	$p = 0.26$

Figure 15. Three ways of asking the same question, and every interval covers zero. Strict rule, the pre-registered primary: -0.046 , $[-0.196, +0.100]$. Lenient rule: -0.042 , $[-0.192, +0.113]$. The third row checks whether a different method returns the same verdict: a binomial GEE clustered on task, on its own scale rather than in runs, -0.443 in log-odds, $[-1.233, +0.348]$, score test $p = 0.26$: the chance of a result at least this strong if the effect being tested is not there at all. The smaller it is, the harder the result is to explain without the effect. $n = 960$ runs over 48 tasks; the first two intervals from 5,000 resamples of tasks, seed 20260819.

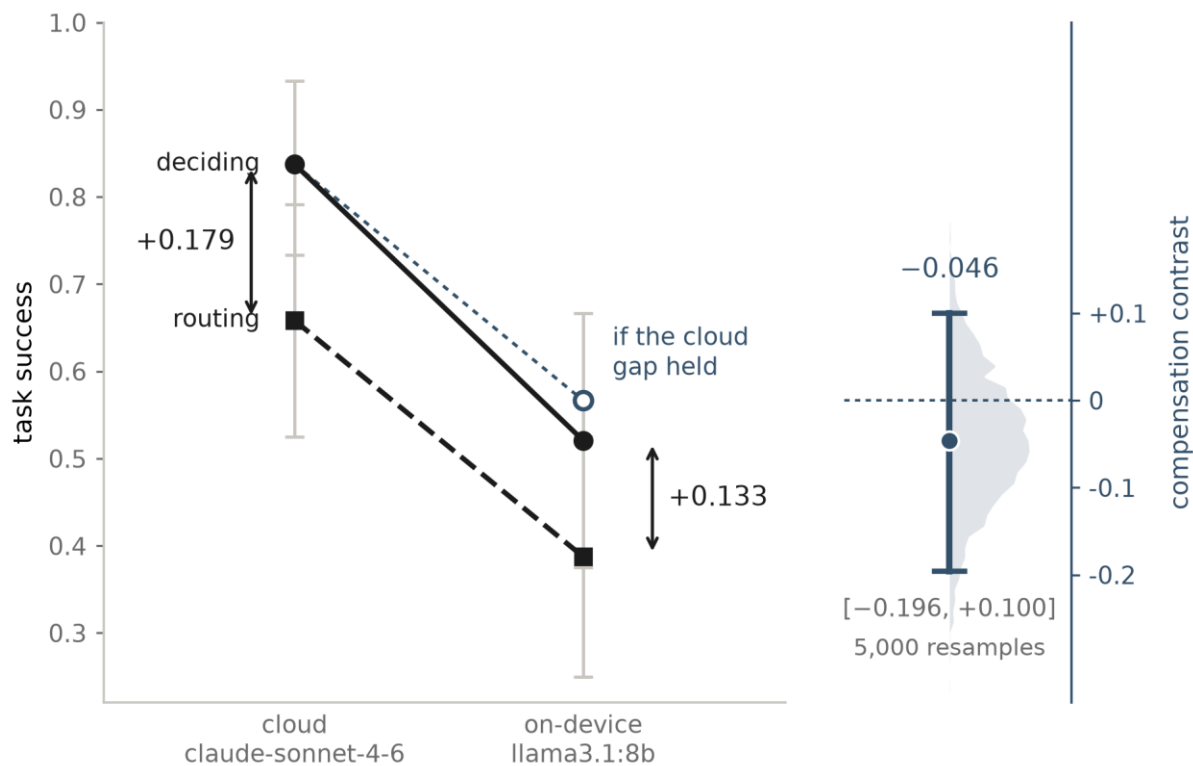


Figure 16. The deciding design's advantage is smaller on the device than in the cloud, and not by enough to tell apart from no change at all. In the left panel each line is one design across the two deployments, and the two arrows are what the design is worth at each: +0.179 in the cloud, +0.133 on the device. The hollow marker is where on-device deciding would have landed if the cloud gap had held exactly; the filled marker below it is where it actually landed. The right panel is the distance between those two markers — the compensation contrast — read against its own axis with zero set at the hollow marker: -0.046, with an interval from -0.196 to +0.100. The grey shape behind that interval is the spread of the 5,000 resampled values the interval was computed from, and the interval is the middle 95% of them. Error bars are 95% intervals on the configuration means. $n = 960$ runs over 48 tasks, seed 20260819.

The interval spans zero, and the second estimator gives the same reading — an interaction that does not separate from zero. The deciding design is worth +0.179 at cloud deployment and +0.133 on-device, and the difference between those two is not distinguishable from sampling noise.

So the deciding design does not measurably compensate for the weaker on-device model. The published record is split on the direction of this effect (§2.3). This study had an 80% chance of detecting a compensation effect of about 0.22 and roughly even odds at 0.15 (§3.9), so what it establishes is a bound rather than an absence: compensation large enough to be worth the deciding design's price is not supported here. That answers the disagreement in §2.3 more usefully than another split verdict would.

§4.3 The two main effects

configuration	task success	decision accuracy	n
C1 deciding / cloud	0.838	0.721	240
C2 deciding / on-device	0.521	0.500	240
C3 routing / cloud	0.658	0.717	240
C4 routing / on-device	0.388	0.417	240

Figure 17. *The four configurations side by side. Task success is the primary measure, with partial completions counted as failure; decision accuracy scores the choice the harness made, whatever came of it. Task success runs 0.838 at C1, 0.521 at C2, 0.658 at C3 and 0.388 at C4. Two rows that differ in one word differ in one factor: C1 against C3 is the design at cloud, C2 against C4 the design on-device, and the difference between those two differences is the quantity this paper reports. Each configuration is 48 tasks × 5 repetitions, 240 runs.*

effect	estimate	95% interval
Deployment (cloud – on-device)	+0.294	[+0.165, +0.423]
Design (deciding – routing)	+0.156	[+0.050, +0.267]
Design at cloud	+0.179	[+0.050, +0.308]
Design on-device	+0.133	[+0.000, +0.271]
Deployment within deciding	+0.317	[+0.167, +0.463]
Deployment within routing	+0.271	[+0.121, +0.421]

Figure 18. *Both main effects separate from zero; only their difference does not. A main effect averages over both settings of the other factor: the design is worth +0.156 [+0.050, +0.267] and the deployment +0.294 [+0.165, +0.423]. The four rows beneath are simple effects — the same comparison made at one setting of the other factor instead of averaged over both. All four order the same way, which is what makes the two averages safe to read. The design on-device has a lower bound of exactly +0.000, and §4.4 returns to it. Same resampling unit and seed as Figure 15.*

Both main effects exclude zero under the primary rule. Where the model runs matters roughly twice as much as how the assistant is built.

Five things qualify those numbers, and they are stated before any interpretation is offered.

§4.4 How much to trust these numbers

The first two do the most damage to what §4.3 reports. First, the design effect is sensitive to which rule was used for the runs that half-finished. The plan fixed both rules before any data existed and required the disagreement between them to be reported.

coding	design	deployment
strict (primary)	+0.156	+0.294
partial counts as failure	[+0.050, +0.267]	[+0.165, +0.423]
lenient	+0.092	+0.292
partial counts as success	[−0.006, +0.198]	[+0.165, +0.421]

Figure 19. Which rule of a partial run is used changes the size of the design effect and leaves the deployment effect alone. Under the pre-registered strict rule, where a run that half-finished counts as a failure, the design is worth +0.156 [+0.050, +0.267]; counting those runs as successes instead takes it to +0.092 [−0.006, +0.198], and that interval crosses zero. Deployment is +0.294 and +0.292. Both rules were written down before any data existed; the paragraph below says where the difference goes.

Counting partial completions as successes cuts the design effect by about two fifths, and its interval crosses zero, while leaving deployment untouched. The reason is where those runs land: the lenient rule raises the two routing configurations by 0.083 each and the two deciding ones by 0.017 and 0.021, and that difference, 0.065, is the whole of the narrowing. The design advantage rests substantially on fully completing tasks that the routing design completes only in part.

Second, the design advantage on-device is the weakest of the four simple effects. Its interval runs from +0.000 to +0.271: the bottom of it sits exactly at no effect, so the data are consistent with the deciding design being worth nothing on the device. Any claim that the deciding design helps specifically on-device is at the edge of what this study can support.

Third, the repetitions are not five independent observations. Temperature was pinned to zero as a control, and it held more completely than the plan assumed.

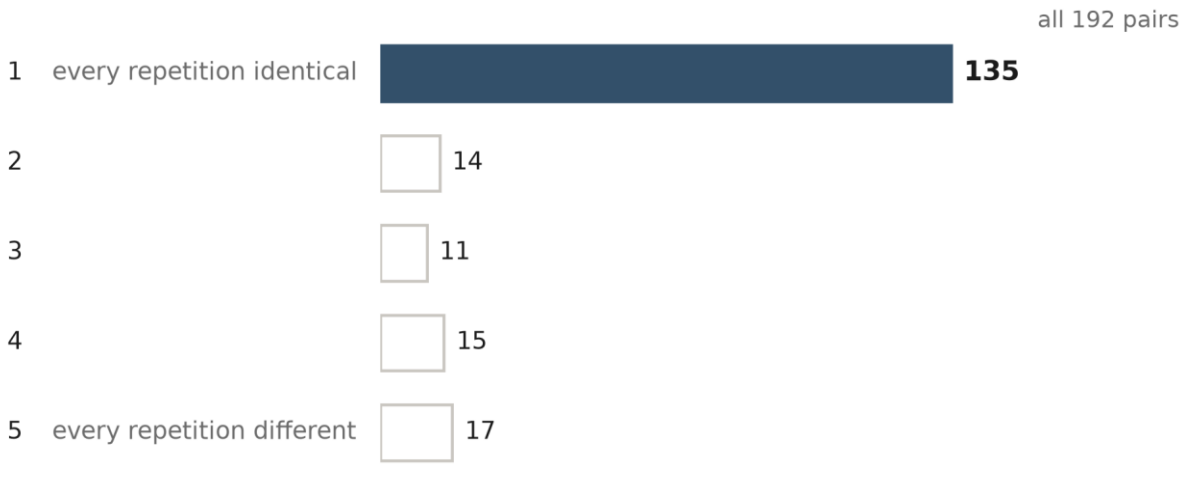


Figure 20. Five repetitions are not five independent observations, which is why the interval is computed on the 48-task unit. In 135 of the 192 task-and-configuration pairs, all five repetitions returned byte-identical replies. Temperature 0 throughout; $n = 192$ pairs over 960 runs.

In 135 of the 192 pairs, all five repetitions returned byte-identical replies. §3.9 gives the consequence: every interval in this section is computed on the 48-task unit, because five identical replies are one observation recorded five times. This figure is how often that is exactly what happened.

Fourth, the pre-registered secondary test returns nothing. It asks whether the design's advantage grows as tasks get harder, using the difficulty each task was given when it was written. The advantage does rise across the five ranks — +0.200, +0.027, +0.127, +0.229, +0.475 — and the gradient is +0.075 per rank, interval $[-0.015, +0.174]$, which spans zero. Four tasks sit at the highest rank and difficulty was ranked within category, so this is a weak test by construction. It is reported because it was pre-registered, not because it settles anything.

Fifth, and not pre-registered: the headline does not depend on the grader. Whether a task is scored by a text check or by the grader was fixed when the task was written (§3.7), so the 48 split cleanly into 15 and 33. Recomputed on each subset, compensation is $-0.147 [-0.507, +0.213]$ on the 15 tasks no grader ever saw, and $0.000 [-0.139, +0.146]$ on the 33 it did. Both cross zero, and removing the grader moves the estimate away from compensation rather than toward it. Fifteen tasks is few and the two subsets are different mixes of category, so this bounds the grader's influence rather than removing it.

Sixth, and not pre-registered: the headline does not depend on the tasks written for this study. Every task records where it came from when it was written (§3.5), so the 48 split into 22 that predate the study and 26 that do not. Compensation is $+0.036 [-0.200, +0.273]$ on the 22 and

$-0.115 [-0.308, +0.077]$ on the 26, so both halves are null. If anything the tasks written for this study point further from compensation than the inherited ones, which is the opposite of what a task set assembled around a result would do; but the difference between the halves is $-0.152 [-0.460, +0.156]$ and covers zero, and the two halves are different mixes of category.

§4.5 What the deciding design does differently

Decision accuracy compares each run's recorded decision against the ideal derived from the request by a fixed rule, with no argument from the design (§3.7). Figure 21 shows something the headline does not.

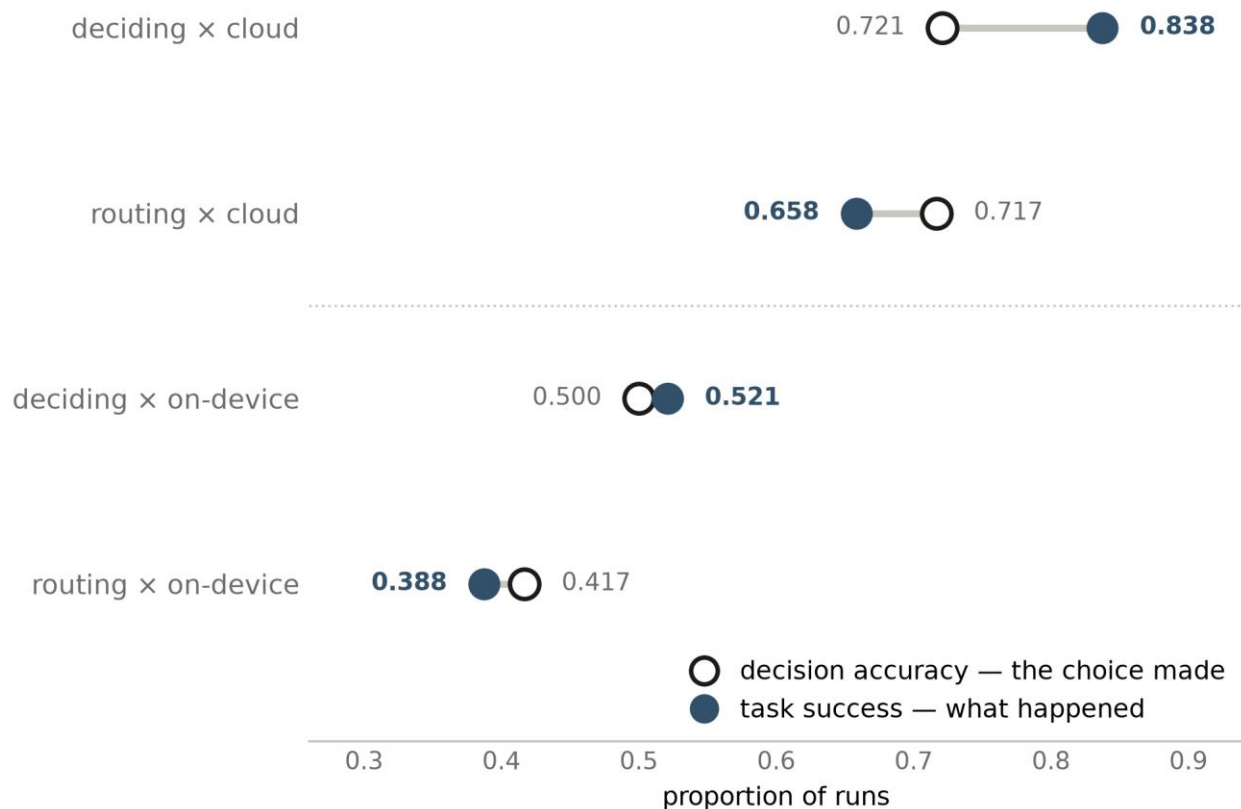


Figure 21. Each row is one configuration, with its two measures on one scale. The hollow marker is decision accuracy, the choice the harness made; the filled marker is task success, what came of it. Both are counts of the 240 runs each configuration has, written as a proportion of them: 0.838 means 201 of those runs did what the request asked for, and 0.721 that 173 of them made the decision the rule called for. At cloud the two designs judge almost identically — 0.721 against 0.717 — and succeed 0.179 apart. On the device they differ on both.

At cloud deployment the two designs decide almost identically — 0.721 against 0.717 — and succeed very differently, 0.838 against 0.658. On-device they differ on both, 0.500 against 0.417.

So the deciding design's cloud advantage is not that it chooses better. It chooses the same, and converts choices into completed tasks more often. So whatever the advantage is, it happens after the decision rather than during it.

The designs differ in three ways (§3.2): the routing design commits in a single model call where the deciding design may revise across up to eight rounds; the deciding design runs every action named in one response where the routing design runs the first and drops the rest; and the deciding design has `ask_user`. Any of the three could be doing the work, and this study cannot tell them apart from the four configurations alone. That question was registered before it was tested, and §5.5 answers it.

Two further counts locate the failure modes, and they are opposites.

configuration	asks when it should not	fails to ask when it should	model-written asks	rule-fired asks
C1 deciding / cloud	40	9	71	0
C2 deciding / on-device	40	25	50	5
C3 routing / cloud	21	16	45	0
C4 routing / on-device	5	40	5	0

Figure 22. *Each design has its own characteristic error, and the two are opposite. A run over-asks when it puts a question to the user on a task whose ideal decision required none, and under-asks when it acts on a task that required one. The deciding design over-asks 40 times in each deployment; the routing design under-asks 40 times on the device. The last two columns separate a question the model wrote from one a rule fired; §4.6 reports what became of that distinction. Counts over 240 runs per configuration.*

The deciding design's characteristic error is over-asking; the routing design's is under-asking. C4 fails to ask on 40 occasions where the ideal required a question and asks unnecessarily only 5 times; C1 and C2 show the reverse. It shows up in the control category (§4.6), where the routing design scores higher precisely because the deciding design asks about requests that needed none.

Inside every configuration, runs whose decision matched the ideal succeeded more often than runs whose decision did not. That has to be true if the decision is what drives the result rather

than the two rising and falling together for some third reason, and it is true in all four configurations.

configuration	success when the decision matched the ideal	when it did not
C1 deciding / cloud	0.890 (n=173)	0.702 (n=67)
C2 deciding / on-device	0.667 (n=120)	0.375 (n=120)
C3 routing / cloud	0.692 (n=172)	0.574 (n=68)
C4 routing / on-device	0.650 (n=100)	0.200 (n=140)

Figure 23. *A wrong decision costs far more on-device than at cloud. Inside every configuration, runs whose decision matched the ideal succeeded more often than runs whose decision did not — but the gap is 0.188 at cloud under the deciding design and 0.450 on-device under the routing design, where a wrong decision is close to fatal. n as marked in each entry; 960 runs in all.*

The gap is widest on-device, where a wrong decision is close to fatal (C4: 0.200) and a right one recovers most of the loss (C4: 0.650).

One prediction fixed in the plan did not survive the data, and it is worth stating plainly. The plan expected the two designs to differ in why they ask: the routing design only through a rule that fires when an action is missing an argument, the deciding design through the model choosing to ask. That is not what happened. The rule fired 5 times in 960 runs, all of them in the deciding design, from a code path both designs share — and every other question in the study, in both designs, was one the model wrote for itself. So the designs do not differ in what makes them ask. The claim was withdrawn before the full run rather than after it, and the counts are reported in Figure 22 as description, with no mechanism resting on them.

§4.6 By category

	cloud		on-device	
	deciding	routing	deciding	routing
trivial (control)	0.875	1.000	0.750	1.000
grounding	1.000	0.650	0.875	0.325
robustness	0.875	0.875	0.500	0.375
reasoning	0.900	0.425	0.500	0.500
action	0.500	0.250	0.375	0.125
clarification	0.875	0.750	0.125	0.000

Figure 24. Two rows carry the section's argument: the control, and the category the criterion failed on. On trivial the routing design scores 1.000 in both deployments and the deciding design does not, which is the control behaving as a control should. On clarification both designs sit at the floor on-device, 0.125 and 0.000, which is why that category failed the gate in Figure 14. $n = 40$ runs in each entry; 960 in all.

The categories were fixed in advance as a taxonomy of request types, not as a difficulty scale. No gradient is fitted across them here. The difficulty each task carries, fixed when it was written, is what the gradient test in §4.4 uses.

Three things the figures support directly.

The control behaves as a control should, and slightly against the deciding design. Trivial was included because the deciding design should not help on single-step requests. The routing design

scores 1.000 in both deployments; the deciding design scores lower in both. With Figure 23, the cause is legible: the deciding design's freedom to ask is a liability where nothing needs asking.

The on-device model's weakness is not uniform. It is nearly absent in trivial and full in clarification, where on-device success is 0.062 across both designs (n=80). The single number "the on-device model is weaker" would conceal a range from no loss to complete loss.

Reasoning is the one category where the routing design does not lose from cloud to on-device (0.425 → 0.500). With 8 tasks and a fully deterministic majority this is not a result; it is flagged as the one pattern a replication should check.

§4.7 The criterion in the full run

§3.11 sets out the criterion and the exception written when the pilot failed it. This is what it did in the full run.

On-device success overall was 0.454 (n=480), comfortably inside the band. Per category: trivial 0.875 (exempt as the control), grounding 0.600, reasoning 0.500, robustness 0.438, action 0.250, and clarification 0.062 (n=80), below the floor.

So the criterion was not met in the full run either, and it failed on the same category the pilot flagged, at almost the same rate. Action, which the pilot marked as the next category at risk at 0.188, rose to 0.250 and sits inside the band without intervention.

The exception therefore holds, and so does its cost: no claim is made about the design within the clarification category at on-device deployment. Nothing else in this section changes, because the criterion asks whether a comparison can exist at all inside a category.

§4.8 What the advantage costs

configuration	median latency	90th percentile	mean cost per run	model calls per run
C1 deciding / cloud	4.54 s	8.62 s	\$0.0169	2.37
C2 deciding / on-device	9.08 s	22.69 s	\$0	2.65
C3 routing / cloud	2.55 s	5.91 s	\$0.0092	1.33
C4 routing / on-device	4.88 s	10.58 s	\$0	1.33

Figure 25. *The deciding design's advantage is bought with time and money, and the figure says how much of each. Median latency 4.54 s against 2.55 s at cloud, 9.08 s against 4.88 s on-device. Mean cost \$0.0169 against \$0.0092 per run at cloud. On-device cost is zero by definition — the model runs on hardware already owned, and energy is not metered here. 240 runs per configuration.*

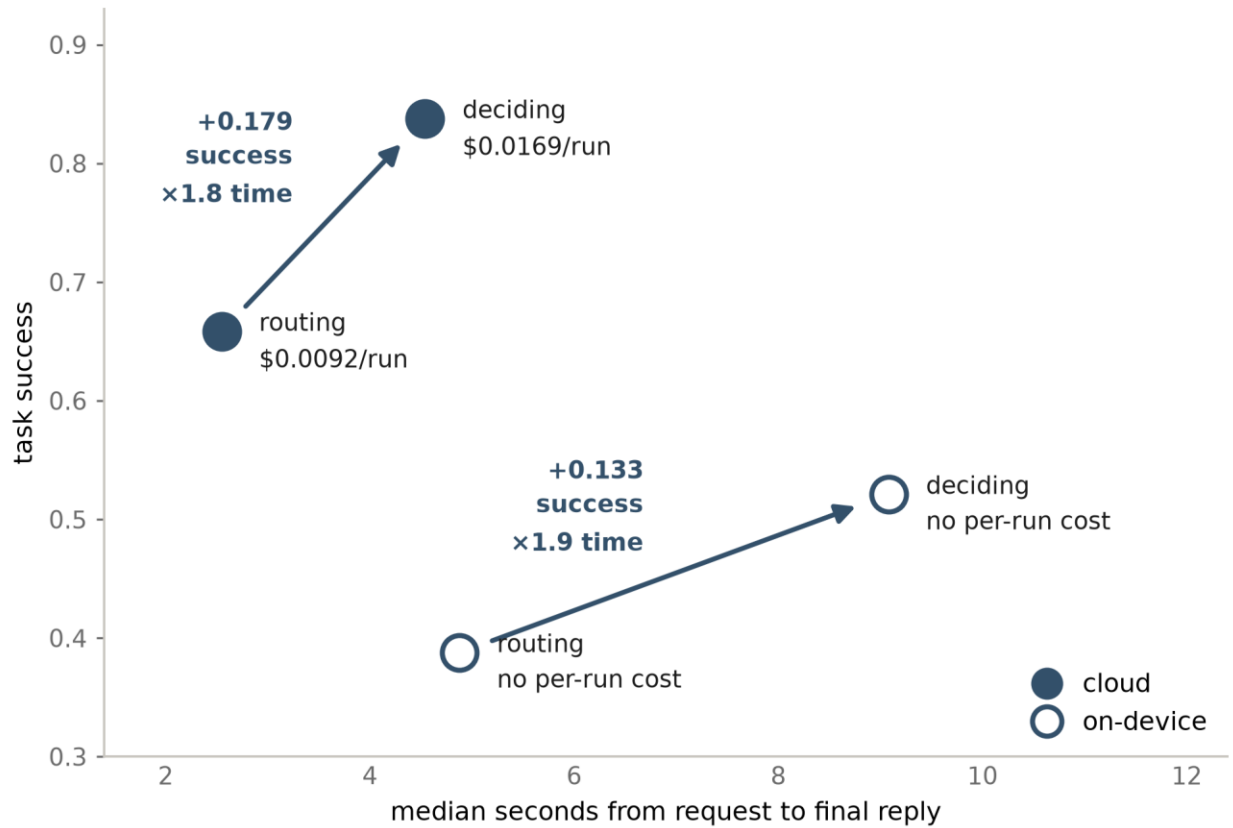


Figure 26. The price is roughly the same in both deployments: about twice the time for about a seventh more tasks completed. At cloud, $+0.179$ task success for $\times 1.8$ the median latency and $1.8\times$ the cost per run. On-device, $+0.133$ for $\times 1.9$ the latency and no per-run cost at all. $n = 240$ runs per configuration.

The advantage is bought, and the figure says with what. At cloud deployment it costs $1.8\times$ the latency and $1.8\times$ the money of the routing design, for $+0.179$ task success. On-device it costs $1.9\times$ the latency for $+0.133$, with the 90th percentile reaching 22.8 seconds — long enough to matter for an assistant meant to be spoken to. The routing design makes exactly one model call in the turn that is scored, by construction (§3.2), so the whole of the deciding design's extra cost is the additional deliberation §4.5 shows is where its advantage comes from. One caveat belongs with the latency. The four configurations ran one after another rather than interleaved, and both on-device configurations ran before both cloud ones (§3.8). Task success is unaffected at temperature zero, but each configuration's latency was measured in a different part of the same afternoon, and the mitigation the plan registered for network variance was not carried out (§3.12).

Exploratory naturalness measures are in Appendix D, and carry no conclusion.

§5 The second study

§5.1 What the first study could not answer

Two things in §4 carry weight that §4's own data cannot support.

The first is a limit of the study's arrangement. Every cloud run used *claude-sonnet-4-6* and every on-device run used *llama3.1:8b*. In the data, cloud deployment and that one model are the same column; so are on-device deployment and the other. Nothing in §4 can tell whether the deployment effect is about where a model runs or about which two models happened to be compared.

The second is a limit of the evidence. §4.5 concludes that the deciding design's advantage is recovery after a decision rather than better decisions, on the grounds that the two designs decide almost identically at cloud deployment and succeed very differently. That is consistent with the conclusion and does not test it. The two designs differ in three respects at once, and §4 separates none of them, so the conclusion could be right and still untested.

This section reports a second study, fixed in writing before it ran, built to close both.

§5.2 What was registered, and when

Study 1's amendment list closed on August 19, 2026, before any of its data existed, and nothing may be added to it (§3.10). The predictions in this section were formed after §4's result was known, so they cannot claim that list's standing and are not filed there. They were fixed before the data in this section existed and after the data in §4 did. That is the difference between this study being registered and the first being pre-registered: both were fixed in writing before their own data existed, and only the first was fixed before any data existed at all.

The predictions were written into a separate document and fixed in the project's version history on September 4, 2026, before any of the runs reported below existed. That record shows when the predictions were written. It does not show what we expected. What it protects against is a prediction changed to fit a result.

The document fixes what would be run and how it would be read: eight models across four families, two cloud and six on-device; three ablations of the deciding design, each removing one property; a check that the rewritten instrument still reproduced the frozen one, required to pass

before any ablation ran; and that every interval would come from resampling the 48 tasks, as in §3.9. The size of difference that would count as no difference was fixed at 0.15 in task success, argued from cost rather than chosen from the data. Moving to the device costs 0.294 of task success, about 29 runs in every hundred (§4.3), and the deciding design costs roughly twice the latency and twice the money of the routing design to run (§4.8). The registration set the line at recovering half of what the device costs: above it, the design's price buys back enough to be worth paying; below it, the price is real and the return is not. Half is a judgment and not a calculation, which is why it was written down before any of this study's data existed. It is not the 0.15 of §3.11, which is a floor on a category's success rate rather than a size of effect. It also fixes the budget at 2,592 runs, so the study could not be extended until it produced a result.

Direction was asserted in one place and refused in another. For the ablation it is asserted: if §4.5 is right that the loop carries the advantage, then removing the loop should remove most of it and removing the ask_user action should not. For the compensation contrast it is refused at every capability gap, because the published record is split (§2.3) and Study 1 does not settle it. The document also fixes what the contrary result would cost — if removing the ability to ask were what removed the advantage, §4.5 would be wrong and §3.2's argument about that action would weaken, and both would be reported rather than dropped. Departures made after this study's data existed are recorded in that same document, dated, in the weaker kind §3.10 describes.

§5.3 What was added

The second study runs 2,592 further runs — 18 configurations of 144 each — on the same 48 tasks, through the same instrument, scored by the same grader against the same standards. The first study's rows were not re-run and not re-scored, and no task was added to the set: a task written after §4's result was known would raise exactly the question the before-and-after-data split exists to answer.

The first gap was that cloud and on-device each held one model. Six were added. One is the second cloud model, *claude-haiku-4-5*. Two models at one deployment vary capability while deployment is held fixed, and that is the only arrangement in this instrument that separates them. Five more models were added on the device: *llama3.2:3b*, *qwen2.5:7b*, *qwen2.5:3b*, *qwen2.5:1.5b* and *mistral:7b*. Within a family, Qwen at three sizes and Llama at two, size changes and the training recipe does not, so a difference along that ladder is attributable to capability. Across families at comparable size: *llama3.1:8b*, *qwen2.5:7b*, *mistral:7b* — capability is roughly held but the recipe varies. Neither arrangement alone separates capability from the training recipe a family uses; together they do.

Capability is not taken from a model's size or its published scores. It is measured on these tasks: the capability gap of a pair is the cloud model's task success under the routing design minus the

on-device model's, over the same 48 tasks. Routing was chosen as the yardstick because it adds no deliberation. §5.4 shows what that choice costs: under that design a stronger cloud model scores below a weaker one. The measure and the rule for pairing were both fixed before the first run, so neither could be picked afterwards to suit a result.

The second gap was that the two designs differ in three respects at once. Three variants of the deciding design were added, each removing exactly one: the loop that shows the model what an action returned, the running of every action named in a single response, and the `ask_user` action.

Each configuration ran three repetitions rather than five. §4.4 reports that 135 of the first study's 192 task-and-configuration pairs returned five identical replies, so a fourth and fifth buy almost nothing. Every interval here is computed on the same 48-task unit as §4.

Every candidate model was screened before the study by putting four requests to it and checking that it could name an action at all, since a design that never acts cannot be told apart from one that cannot act. *phi3:3.8b* was excluded: the request itself was refused. *qwen2.5:1.5b* named an action on two of the four and answered the other two in prose without acting. It was kept deliberately, because excluding a model for being weak removes the bottom of a ladder built to go from weak to strong.

The screen was not sufficient, and the runs showed it. It offered two actions; the study offers 21. Given 21, *mistral:7b* named an action in none of its 288 runs across both configurations, answering instead by describing actions it had not taken — "I have saved the WiFi password," where nothing was saved. With no action ever named, the loop never runs, so its two configurations are the same procedure under two labels. Its rows are reported, and its difference between designs is not a test of the design. The fault is in this study's screening rule, and it costs two of the 18 configurations.

Two of the three variants required a part of the instrument to be rewritten outside the freeze, because the properties they remove sit inside one function. The rewrite was therefore run with every property left in place and compared against the unmodified instrument on the same machine in one session, matching it on the reply text, the actions called and the number of iterations for all 48 tasks but one: the task that asks for the time, whose reply changes whenever a minute passes. Appendix C records the check and the two departures made after this study's data existed.

§5.4 Compensation across 12 pairings

Six on-device models against two cloud models give 12 pairings, and each pairing yields one compensation contrast: how much the deciding design is worth on-device, minus how much it is worth in the cloud (§3.9). A positive value means the deciding design buys more on the weaker deployment. A negative value means it buys less.

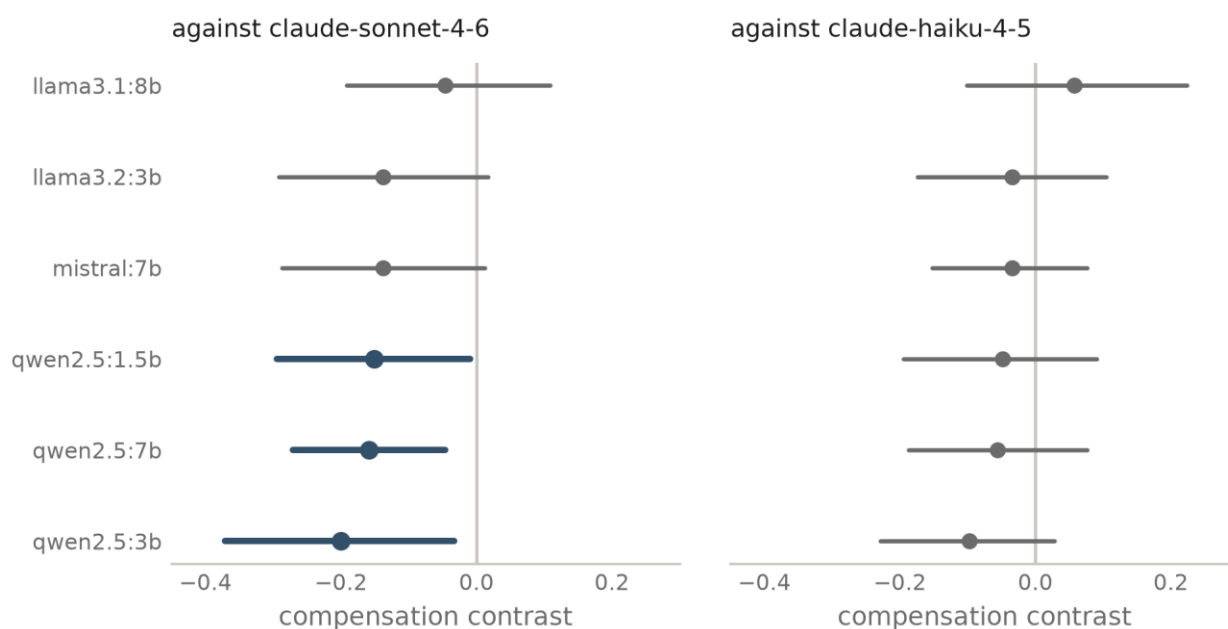


Figure 27. Across 12 model pairings the contrast is negative in 11, and excludes zero in three. Six on-device models against two cloud models. Against *claude-sonnet-4-6* the intervals for *qwen2.5:1.5b*, *qwen2.5:3b* and *qwen2.5:7b* exclude zero, all three negative; against *claude-haiku-4-5* no interval excludes zero. *mistral:7b* never named an action and does not test the designs (§5.3). $n = 144$ runs per configuration ($48 \text{ tasks} \times 3 \text{ repetitions}$), except the two Study 1 configurations, which carry 240. Intervals resample the 48 tasks; 5,000 resamples, seed 20260819.

Nine of the 12 intervals cover zero. The three that do not are all negative, and all three pair an on-device model against *claude-sonnet-4-6*: *qwen2.5:3b* at $-0.200 [-0.371, -0.033]$, *qwen2.5:7b* at $-0.158 [-0.271, -0.046]$ and *qwen2.5:1.5b* at $-0.151 [-0.294, -0.010]$, each over the same 48 tasks. Against *claude-haiku-4-5*, no pairing separates from zero in either direction.

The pairing the first study reported, *llama3.1:8b* against *claude-sonnet-4-6*, is the least negative of the six that use that cloud model: $-0.046 [-0.192, +0.108]$. Against *claude-haiku-4-5* the same on-device model gives $+0.057 [-0.101, +0.224]$, the only positive contrast in the 12. So §4 did not happen on an unfavourable pairing. Of the six on-device models, the one it used gives the least negative contrast against *claude-sonnet-4-6* and the only positive one against *claude-haiku-4-5*, and neither separates from zero. The interval here is recomputed by the second study's own

analysis, which draws its 5,000 resamples independently of §4.2's; the two differ in the third decimal for that reason and no other.

Of the 12 contrasts, 11 are negative. Across six on-device models from three families and two cloud models, the deciding design is not measurably worth more on the device than in the cloud, and where the contrast separates from zero it is worth less.

The registration fixed one further test and said in advance what each of its two possible results would mean. Compensation is fitted as a straight line against the capability gap — the cloud model's task success under the routing design minus the on-device model's, so the gap is a property of the pairing — with the interval computed on the same 48-task unit as every other interval in this paper. The slope is how much the compensation contrast changes for each unit of capability gap. An interval clear of zero would mean compensation grows as the two models are pulled further apart, which would make §4's null a fact about *llama3.1:8b* being too close to the cloud model. An interval covering zero would mean it does not grow, which would make §4's null a fact about the two designs.

The slope is +0.307 with a 95% interval of $[-0.039, +0.715]$, so the second of those is what occurred. The six pairings the line rests on span a gap of 0.279, from -0.008 to $+0.271$, so +0.307 amounts to about 0.086 of compensation between the closest pairing and the widest. The gap can also be measured by decision accuracy, which scores the action the model chose and never passes through the reply; on that measure the slope is +0.242 $[-0.099, +0.705]$ and the reading is the same.

The line rests on six points rather than 12. Changing the cloud model shifts every pairing's contrast and every pairing's gap by the same amount, so the second cloud model redraws the same line rather than adding new points to it.

The contrasts run negative for a reason that has nothing to do with capability.

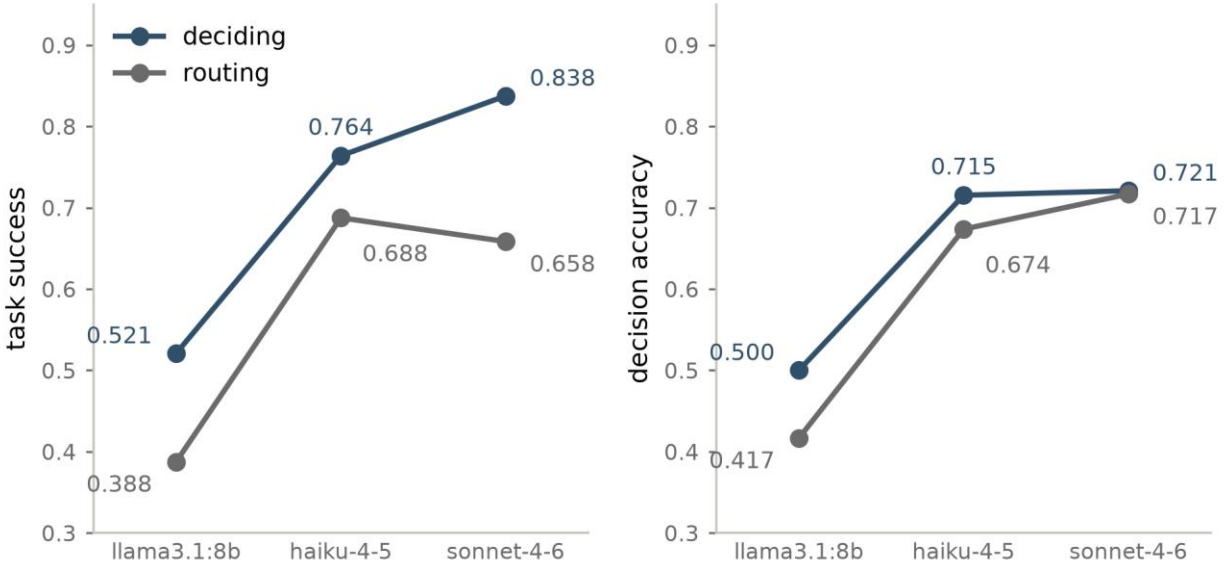


Figure 28. *Task success under the routing design does not track model capability. Under the deciding design it rises with the model: 0.521 for llama3.1:8b, 0.764 for claude-haiku-4-5, 0.838 for claude-sonnet-4-6. Under the routing design it does not, the weaker cloud model scoring 0.688 against the stronger model's 0.658. The right panel is the control: decision accuracy never passes through the reply, and it orders all three correctly under both designs. $n = 240$ runs per configuration for the two Study 1 models, 144 for claude-haiku-4-5.*

Under the routing design, task success does not rise with the model: the stronger of the two cloud models scores below the weaker one, and both run in the cloud, so deployment cannot account for it. Decision accuracy is the one measure that never passes through the reply, and it orders all three models correctly under both designs. So the models are ordered correctly whenever the reply is taken out of the scoring, and what the routing design gets wrong is not which model decides better but what happens to the reply afterwards.

The cause is the routing design's defining rule from §1.2: what an action returns is what the user reads. A model that correctly reaches for the arithmetic action sends back a bare number, and that number is the whole of the reply.

ACTING COSTS TASK SUCCESS

for both cloud models, under routing

		when it acted	when it did not
claude-sonnet-4-6		0.471 (153)	0.989 (87)
claude-haiku-4-5		0.538 (78)	0.864 (66)

The bar is task success on the runs that acted.

AND THE STRONGER MODEL ACTS MORE OFTEN

how often each names an action, in the three categories where they differ

	success when acted	sonnet acts	haiku acts
grounding	0.263	47.5%	25%
clarification	0.000	25%	12.5%
robustness	0.714	35%	12.5%

Overall 64% of 240 runs against 54% of 144. On reasoning both act on 75%, and on trivial and action both act on every run, so the whole of the difference is in these three.

Figure 29. Under the routing design, acting costs task success, and the stronger model acts more often. The reply the user reads is what the action returned, so a run that names an action is a run whose reply the model did not write. The three categories below are the only ones where the two models differ in how often they act: on reasoning both act on 75% of runs, and on trivial and action both act on every run. Strict rule; $n = 240$ routing runs for claude-sonnet-4-6, 144 for claude-haiku-4-5.

So the routing design penalizes a model for using an action well, and penalizes the stronger model harder because it reaches for actions more often. Its routing score falls. The deciding design's cloud advantage is measured against that fallen score, so the cloud advantage grows. Compensation subtracts the cloud advantage from the on-device one, and three pairings are pushed negative by that subtraction alone.

Every interval so far has answered one question: is there an effect? An interval that covers zero has failed to find one, which is not the same as showing there is none — it also covers values large enough to matter. The opposite question needs its own test. An equivalence test asks whether the whole interval sits inside a margin small enough to ignore (Lakens, 2017), which is why the margin has to be fixed in advance: the 0.15 of \$5.2. It is judged on a 90% interval rather than the 95% ones used elsewhere here. The claim fails if the interval spills past either end of the margin, and it can only spill past the end the true effect lies beyond. A 90% interval leaves 5% beyond each end, which gives the equivalence claim the same one-in-twenty chance of being

wrong as a 95% interval gives an ordinary one. The registration fixed all of this before the study ran.

Two of the 12 pairings meet the equivalence margin of 0.15 that §5.2 fixed from the cost of the deciding design: against *claude-haiku-4-5*, *llama3.2:3b* at $[-0.146, +0.083]$ and *mistral:7b* at $[-0.132, +0.062]$, both 90% intervals lying inside ± 0.15 . One of the two is *mistral:7b*, whose two configurations run the same procedure (§5.3) and so cannot settle anything about the designs. One pairing in 12 is therefore precise enough to say compensation is absent for that pairing rather than only undetected. The registration recorded 0.15 as a margin this study was unlikely to reach, and it was reached once.

§5.5 Where the advantage comes from

Each variant is the deciding design with one property taken out and nothing else changed. Each variant is measured against the routing design at its own deployment, the same comparison §4 made for the full deciding design. The full design's advantage and each variant's are therefore the same quantity, and one axis holds all of them.

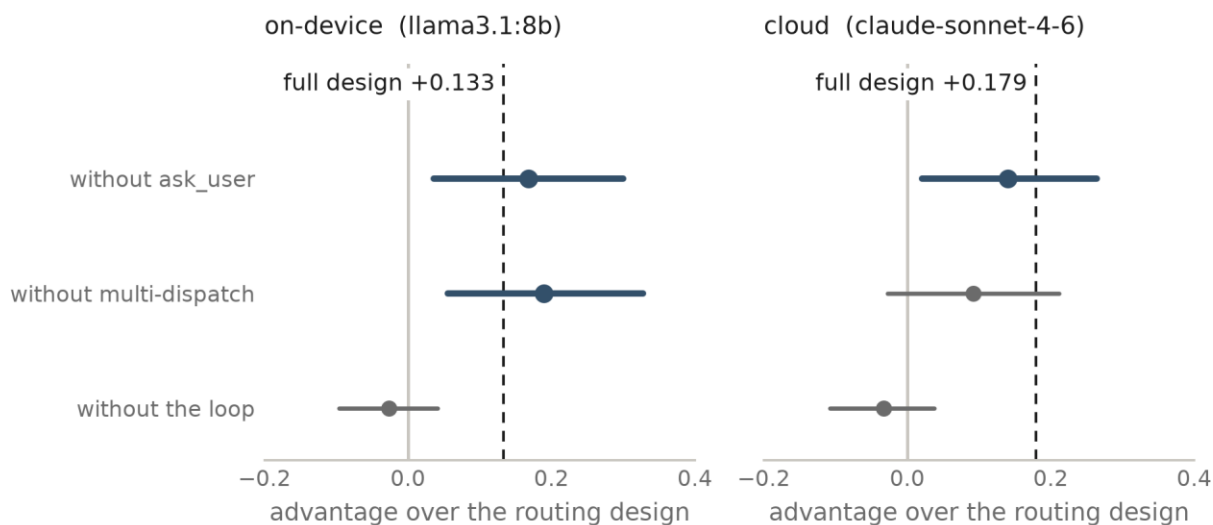


Figure 30. *Only the loop is doing the work. Removing the loop removes the whole advantage: -0.026 $[-0.096, +0.042]$ on-device and -0.033 $[-0.108, +0.037]$ in the cloud, against $+0.133$ and $+0.179$ for the full design. Removing the running of every action named in one response, or the `ask_user` action, leaves it standing: $+0.189$ and $+0.168$ on-device, $+0.092$ and $+0.140$ in the cloud. $n = 144$ runs per variant; 5,000 resamples, seed 20260819. The rewritten instrument matched the frozen one on 47 of the 48 tasks, the exception being the task that asks for the time.*

Removing the loop removes the advantage completely. On the device the full design is worth $+0.133$ $[-0.004, +0.275]$ over the routing design and the variant without the loop is worth -0.026

$[-0.096, +0.042]$. In the cloud the same two numbers are $+0.179 [+0.050, +0.308]$ and $-0.033 [-0.108, +0.037]$. Two deployments, two models, and the advantage lands within noise of zero in both.

That direction was fixed before either variant ran. §4.5 concluded that the deciding design's advantage is recovery after a decision rather than a better decision, and §5.2's registration turned that conclusion into a prediction: removing the loop should remove most of the advantage, and removing the `ask_user` action should not. Both parts held. Removing `ask_user` leaves $+0.168 [+0.036, +0.300]$ on the device and $+0.140 [+0.021, +0.264]$ in the cloud. Removing the running of every requested action in one response leaves $+0.189 [+0.056, +0.328]$ and $+0.092 [-0.028, +0.211]$; no direction was predicted for that one, because nothing in the first study spoke to it.

Both of those variants score at or above the full design at their own deployment, but the intervals overlap it heavily and no test was run on the difference, so what the data supports is that the advantage survives their removal rather than that it grows.

Had `ask_user` been the property that carried the advantage, §4.5's conclusion would have been wrong and §3.2's argument that `ask_user` does not reintroduce a difference in available actions between the designs would have been weakened.

One comparison lies beyond the registration. Each property has a cost — the advantage with it, minus the advantage without it — and that cost can be compared across the two deployments. For the loop the difference is $-0.053 [-0.215, +0.111]$, so the loop is worth about the same whichever model is behind it. For `ask_user` it is $-0.074 [-0.221, +0.089]$. For running every requested action in one response it is $-0.143 [-0.261, -0.022]$, the one interval of the three that excludes zero. That one says the property is worth having with a strong model and not worth having with a weak one. It is the same kind of effect this study set out to measure — a property of the harness being worth more at one deployment than the other — pointing the other way: compensation would mean the deciding design buys more on the device, and this says one of its three properties buys less. Three such comparisons were made and only one cleared zero, so it is offered as something a replication should check rather than something this study settles.

§5.6 What the second study does not settle

It adds models and it isolates a mechanism. It does not add tasks: the same 48 produce every number in this section, and §7 states what that costs.

The ablation ran at one model per deployment, *llama3.1:8b* and *claude-sonnet-4-6*, rather than across the ladder. The finding that the loop carries the advantage therefore rests on two points where the compensation contrast rests on 12, and nothing here tests whether the loop still carries it at the weak end of the ladder. The two cloud models also come from one provider, so "cloud" in this section means two models from the same family.

The registered slope is fitted against a capability gap computed from routing-design scores, and §5.4 shows that design putting two cloud models in the wrong order. The six gaps against *claude-sonnet-4-6* span -0.008 to $+0.271$. The lowest of them, -0.008 , belongs to *qwen2.5:3b*, a three billion parameter model on a laptop, scoring fractionally above the cloud model under the routing design. What it records is the cloud model being penalized for acting. Measuring the gap by decision accuracy avoids the reply and returns the same reading (§5.4). The line includes *mistral:7b*, which cannot test the designs; dropping it moves the slope to $+0.358$ $[-0.076, +0.784]$ and leaves the interval covering zero. Still, the interval covering zero means no relationship was detected across a narrow range on a compressed scale, which is a weaker statement than there being none.

Of the 12 pairings, 11 say that compensation was not detected. One says it is absent: *llama3.2:3b* against *claude-haiku-4-5*, whose interval fits inside the 0.15 margin. The second pairing that meets the margin is *mistral:7b*, which cannot test the designs at all (§5.3). Not detected and absent are different claims, and this study supports the second one once in 12.

Nothing here varies the routing design's reply rule. The rule depresses a strong model's score, and it is shown; but whether a routing design that composed its own reply would close the gap is not, because no such design was built or run.

§6 Discussion

§6.1 What the harness buys back

§1 asked what remains for a builder who has ruled out the cloud. If no request may leave the device, sending the hard ones elsewhere is not available, and what is left is how the assistant is built around the model. This study's answer is that the harness returns a little under half of what the device costs, and that none of that return is specific to the device.

Running on the device rather than in the cloud costs 0.294 in task success on these tasks. Building the deciding design rather than the routing design returns +0.133 of that on the device. Read alone, those two numbers say that a better harness partly rescues a weaker model. The second study shows that reading is wrong: the same design returns +0.179 in the cloud, where the model needs no rescuing, and across 12 pairings of six on-device models against two cloud models, the on-device return is never measurably larger than the cloud return.

So the deciding design is not a remedy for a weak model. It is worth building at either deployment, and on the device — where a builder would most want it to pay — it pays slightly less. Moving to the device and building the deciding design gives a builder a loss of 0.294 and a gain of 0.133 that the data does not tie together; the gain was available without the move.

The claim is narrower than "design cannot compensate for capability." Two designs were compared and a third could behave differently. What is ruled out is that the deciding design — the one that acts, sees the result, and can act again — recovers what a weaker model loses. Among the designs in the published record, that was the one with the strongest reason to.

§6.2 If only one thing could be built

The deciding design differs from the routing design in three ways, and one of them accounts for the whole difference in task success. Removing the loop that shows the model what its action returned takes the advantage from +0.133 to -0.026 on the device, and from +0.179 to -0.033 in the cloud. Removing the ability to ask the user a question, or the ability to run several actions from one response, leaves it standing. That direction was written down before either variant ran, which makes it a test rather than an observation.

A builder with time to build one thing should spend it on letting the model see what its action returned and act again. Running several actions per response and asking the user questions do not produce the advantage.

The loop is also what the design's price buys. Those extra seconds are the model taking another turn after seeing a result, so a builder cannot cut them and keep the advantage.

§6.3 The routing design penalizes its own model

Under the routing design the stronger of the two cloud models scores worse: *claude-haiku-4-5* reaches 0.688 in task success and *claude-sonnet-4-6* reaches 0.658. Both run in the cloud, so deployment does not explain it. Under the deciding design those models order by capability as expected, and decision accuracy orders them correctly under both designs. The cause is the rule that defines the routing design: what an action returns is what the user reads. Under that rule acting is a gamble — *claude-sonnet-4-6* succeeds on 0.471 of the runs where it acts and 0.989 of those where it does not — and the stronger model acts more often, on 63.8% of runs against 54.2% (Figure 29).

What the grader read was exactly what a user would read, so the low score is a fact about the design and not about how strictly it was marked. And because it is the model that is better at deciding to act which triggers that rule more often, a routing design's task success can fall as its model improves (§5.4). This is not a weakness of the version built here: the routing designs this study drew from the literature (§2.2) pass an action's return to the user in the same way.

§6.4 What is settled, and what remains open

The published work is split on whether a more deliberate harness helps a weaker model more (§2.3), and this study does not resolve that split. It replaces an argument with a bound: on these tasks the interval only reaches +0.100, so compensation large enough to pay for the deciding design is not supported. What has changed is that both of its sides now carry a number: whatever the effect is, it lies between −0.196 and +0.100, where before there were two studies pointing opposite ways and no range at all.

Four things this study could not do mark where the next one starts.

- Its 48 tasks were written for it rather than drawn from what people actually ask an assistant, so the result rests on a workload argued to be representative rather than sampled; replaying these designs against real assistant logs would test that directly.

- Its ablations ran at one on-device model and one cloud model, so of the six models on the ladder the loop's role is established at one. Running them across the ladder would show whether the loop still carries the advantage where the model is weakest.
- An extra ablation could have been added to have the routing design generate its own replies.
- And the capability gap the 12 pairings are ranked on is measured under the routing design, which §6.3 shows ordering two cloud models backwards; a gap computed from a design that does not distort it would put the slope on firmer ground.

§7 What this study does not show

A controlled experiment buys its precision by holding things fixed, and everything held fixed is a boundary on what the finding travels to.

The task set is one set, written by one person. Forty-eight tasks decided every number in this paper. Of them, 22 predate the study and could not have been shaped by its hypotheses, §4.4 recomputes the contrast on those 22 and on the 26 written for this study: both are null, and the two halves cannot be told apart. That bounds the concern without removing it. The two halves are not matched on category — one task predates the study in the action category against seven written for it — so part of the difference between them is category mix. A finding that holds across six models and one task set remains, strictly, a finding about that task set.

The tasks work across a small world. Files, notes, timers, and the machine's own state, through 21 actions. That is enough to test which action is chosen and in what order, which is what the tasks were built for, and it is not enough to say anything about breadth. An assistant with two hundred actions faces a selection problem this study did not pose.

Each design is one implementation of its family. The routing design is 87 lines and the deciding design is 129. Both are drawn from published families (§2.2), but nothing here separates a property of the family from a property of these two files. A different loop, or a different router, could behave differently and this study would not know.

The conversation is scripted. Of the 48 tasks, 38 are a single request. The 10 that are longer run to a fixed script, so what the user says next never depends on what the assistant just said. That makes the comparison clean, because both designs receive identical inputs. It also means no result here describes a conversation that adapts, which is what most real ones do.

Three things about the run cannot be pinned down: the quantization, the cloud model's identity, and the time of day. The on-device model runs at four bits per value because that is what fits in 6GB, so this study cannot separate what a smaller model costs from what quantization costs. The cloud model is named by an alias the provider controls rather than by a fingerprint of the weights, so the cloud half is reproducible only as far as that name keeps pointing at the same model. And every configuration ran inside a single two-hour window, one after another rather than interleaved, so the latency figures in §4.8 describe that afternoon and not a typical one.

The null is bounded loosely. The power figures in §4.2 mean a compensation smaller than about 0.22 could be present here and still have been missed, and the procedure that produced them is mildly generous (§3.9), so even that is a little flattering. The margin argued from what the deciding design costs is 0.15. Declaring equivalence at that margin needs the whole 90% interval

to lie inside ± 0.15 , and this one, $[-0.167, +0.079]$, falls outside it on the low side. So this entails that compensation large enough to be worth buying is not supported, not that compensation is absent.

One category is excluded from the design claim. The pre-registered validity criterion was not met for clarification at on-device deployment (§3.11), where both designs sit near zero. No claim is made about the harness design within that category at that deployment, and the exception was written before the full run.

Two of the second study's 18 configurations do not test anything. *mistral:7b* never named an action, so its two configurations run the same procedure under two labels (§5.3). The screening rule that admitted it was this study's own, and it was too small: four requests and two actions, where the study offers 21. A screen that cannot distinguish a model which acts from one which describes acting is too weak, and it cost two configurations.

The grader has not been checked against a person. The plan called for a blind human sample to be rated alongside the grader so that agreement between them could be reported. That was not done. What was done instead is narrower: a second model graded the same transcripts blind and agreed at $\kappa = 0.73$ — agreement well beyond what chance alone would produce (Cohen, 1960) — and on the 33 tasks a grader scored, the contrast is $+0.000 [-0.139, +0.146]$ under the grader of record against $-0.055 [-0.212, +0.109]$ under the second. The estimate moves; the reading does not, which is what a check on the grader can establish. It does not show that either grader agrees with a person. An earlier attempt used the on-device model as the second grader, chosen because its bias should run the opposite way, and was abandoned when four of its first six verdicts came back in a form that could not be read as a verdict. That shows the finding does not depend on which model graded it. It does not show that either model agrees with a person.

Nine departures were made after data existed. They are listed in Appendix C and are of the weaker kind §3.10 describes: recorded when they happened, not protected by anything. One of them changed a number in this paper. The interval on §5.4's slope was first computed by drawing the six pairings themselves, rather than the 48 tasks the registration named. With only six points, a draw that happens to pick pairings of similar capability stacks them into a near-vertical line, and some of those draws returned slopes past $+1$. Redrawn on the registered unit — 48 tasks with replacement, every pairing's contrast and every pairing's capability gap recomputed on them, the line refitted so the whole fit moves together — the interval goes from $[+0.092, +0.595]$, which excludes zero, to $[-0.039, +0.715]$, which does not. The point estimate is $+0.3075$ either way. The correction was made before §5.4 was written, no number from the first version appears anywhere in this paper, and the departure ran in the direction that would have flattered the result.

What the study does show is bounded by all of the above and survives it: across two harness designs, six models spanning three families, and 48 tasks whose standards were written before any of them ran, the deciding design's advantage does not grow as the model gets weaker.

§8 Conclusion

This study asked how much a more deliberate harness buys back when a personal assistant has to run on the user's own device. It buys back less than the device costs, but none of what it buys back is specific to the device, where there is nothing to buy back. The deciding design is worth building. It is not a remedy for a weaker model.

The compensation contrast — how much the deciding design is worth on the device, minus how much it is worth in the cloud — came out at -0.046 , with a 95% interval from -0.196 to $+0.100$ over the first study's 48 tasks and 960 runs. Widening the comparison to 12 pairings of six on-device models against two cloud models, over 2,688 runs, left it there: 11 of the 12 contrasts are negative, three separate from zero, and all three of those are negative. The two studies come to 3,552 runs together, the difference being the ablation. Both fixed their analysis in writing before their data existed, and every departure from those documents is recorded with its date and what it cost.

For a builder who has ruled out the cloud — for privacy, for cost, or to work offline — the capability lost by moving to the device is something to plan for rather than something to design around. The harness is still worth the effort: the deciding design is worth $+0.133$ on the device, and what produces that is the loop that lets the model see what its action returned and act again, since removing the loop removes the whole advantage at both deployments. The choice of harness matters in one more way that has nothing to do with the device: under the routing design a stronger model scored worse than a weaker one (§6.3). Spend the effort on the loop, budget for the capability, and do not let an action's output stand as the reply.

Acknowledgements

I sincerely thank my supervisor for his support and encouragement throughout this project. I am also grateful for his willingness to engage with the completed research and assist me in preparing for its oral defense.

Declaration of assistance from artificial intelligence

Two AI tools were used in this work. ChatGPT (OpenAI) was used in June 2026 and again on five occasions between August 24 and September 4, 2026; no record of those sessions was kept. Claude Code (Anthropic), models claude-opus-5 and claude-fable-5, was used across five sessions between August 15 and September 13, 2026; those transcripts are retained in full and are submitted with this paper.

What the AI tools did:

- Drafted the Study 1 pre-registration document from the design given to them (ChatGPT, June 2026).
- Reviewed drafts and returned lists of gaps in them; three of the gaps it named — more models, an ablation, and a robustness check — set the agenda for the second study (ChatGPT, August–September 2026).
- Wrote the analysis and experiment-running code in tools/.
- Executed the experimental runs.
- Proposed the statistical model — a mixed-effects logistic regression clustered on task — and, when the software would not fit it, the GEE used in its place.
- Suggested the 0.15–0.85 validity criterion.
- Proposed that the second study be registered in writing and tagged before any of its data existed, and drafted that document.
- Proposed the capability ladder within one model family, the three ablations and their registered predictions, and the equivalence margin.
- Found and corrected an error in the registered slope's interval, which moved it from excluding zero to covering zero.
- Generated the figures from the result files.
- Reviewed drafts section by section, returning lists of defects, and supplied example versions of sections.
- Applied mechanical corrections to the manuscript: spelling of numbers, decimal places, italics, fonts and repeated phrasing.

The retained Claude Code transcripts are submitted as supporting material and record each of these. The submission guidance does not state a format for such records, and a query sent to the

organizing committee on September 7, 2026 was unanswered at the time of submission; the record is therefore provided as one exchange for each item above, with the complete transcripts retained and available in full on request.

References

- Chen, J., Shen, J., Kang, H., Hong, Z., Jiang, Q., Bose, S., Zhang, Y., Leng, L., Vyas, A., Mao, L., Ouyang, S., Zhou, K., & Qin, L. (2026). AgentSpec: Understanding embodied agent scaffolds through controlled composition. *arXiv:2606.14674*. <https://arxiv.org/abs/2606.14674>
- Cohen, J. (1960). A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1), 37–46. <https://doi.org/10.1177/001316446002000104>
- Fang, W., Han, D.-J., Yuan, L., Chen, E., & Brinton, C. (2025). Bridging on-device and cloud LLMs for collaborative reasoning: A unified methodology for local routing and post-training. *arXiv:2509.24050*. <https://arxiv.org/abs/2509.24050>
- Field, C. A., & Welsh, A. H. (2007). Bootstrapping clustered data. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 69(3), 369–390. <https://doi.org/10.1111/j.1467-9868.2007.00593.x>
- Lakens, D. (2017). Equivalence tests: A practical primer for t tests, correlations, and meta-analyses. *Social Psychological and Personality Science*, 8(4), 355–362. <https://doi.org/10.1177/1948550617697177>
- Liang, K.-Y., & Zeger, S. L. (1986). Longitudinal data analysis using generalized linear models. *Biometrika*, 73(1), 13–22. <https://doi.org/10.1093/biomet/73.1.13>
- Patil, S. G., Zhang, T., Wang, X., & Gonzalez, J. E. (2023). Gorilla: Large language model connected with massive APIs. *arXiv:2305.15334*. <https://arxiv.org/abs/2305.15334>
- Søgaard, A., Hershcovich, D., & de Lhoneux, M. (2023). A two-sided discussion of preregistration of NLP research. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, pp. 83–93.
- Starace, J. (2026). Scaffold effects on GAIA: A controlled comparison. *arXiv:2606.08529*. <https://arxiv.org/abs/2606.08529>
- Vaccaro, M. (2026). Preregistration for experiments with AI agents. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, PMLR 306. *arXiv:2606.11217*. <https://arxiv.org/abs/2606.11217>
- Xu, H., Li, C., Ma, X., Ou, X., Zhang, Z., He, T., Liu, X., Wang, Z., Liang, J., Chu, Z., Liu, R., Mu, R., Tu, D., Liu, M., & Qin, B. (2026). The evolution of tool use in LLM agents: From single-tool call to multi-tool orchestration. *arXiv:2603.22862*. <https://arxiv.org/abs/2603.22862>

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*.

Zhang, Y., Hu, M., Lin, Z., Fan, X., Xie, F., Fang, Z., Yang, J., Zhu, W., Chen, Z., Lv, C., & Chen, Z. (2026). Hera: Learning long-horizon coordination for device-cloud collaborative LLM agents. *arXiv:2605.24598*. <https://arxiv.org/abs/2605.24598>

Appendix A — The task set

All 48 tasks, as run.

This appendix is generated by `tools/build_appendix_a.py` from `apparatus/tasks/tasks.json` and from nothing else, so it cannot drift from the file the experiment ran against. That file's SHA-256 is

0a3ae35b1ac8343b13880f01393b82960ff6bfe2e5c5927739424b623c1d85b3

and the same hash is carried on every one of the 960 data rows (§3.4).

Each task carries the request or requests put to the assistant, what counted as success, a difficulty from 1 to 5 fixed when the task was written, and where the task came from. The success criterion is reproduced as the scoring code applies it, which is why some read as a pattern or a list of strings rather than as a sentence. §3.5 describes how the set was assembled; §4.4 reports the contrast recomputed on the tasks that predate this study against those written for it.

category	tasks	what it is for
trivial	8	the control — requests both designs should handle identically and both get right
reasoning	8	requests needing a conclusion drawn before acting
action	8	choosing and sequencing actions
clarification	8	requests missing a detail the assistant needs
grounding	8	something said earlier is needed later
robustness	8	requests that are malformed, impossible, or outside what the assistant can do

trivial (8 tasks)

id	difficulty	request	success criterion	origin
----	------------	---------	-------------------	--------

triv-01	1	what time is it	reply matches <code>\b\d{1,2}:\d{2}\b</code>	predates this study, unchanged
triv-02	1	what's today's date	states today's calendar date	predates this study, unchanged
triv-03	2	open the calculator	issues a launch for the Calculator app; the sandbox records the request rather than starting a process	predates this study, unchanged
triv-04	2	set a timer for five minutes	sets a timer for 5 minutes	predates this study, unchanged
triv-05	1	turn the volume down	issues a volume-down action; the sandbox records it	written for this experiment
triv-06	2	show me my notes	lists the notes currently stored, or states there are none	written for this experiment
triv-07	1	what's 12 times 8	reply contains 96	written for this experiment
triv-08	2	cancel my timer	cancels the timer, or states there is none to cancel	written for this experiment

reasoning (8 tasks)

id	difficulty	request	success criterion	origin
reason-01	1	A train leaves at 3:50 and the trip takes 95 minutes. What time does it arrive?	reply contains 5:25	predates this study, unchanged

reason-02	2	Which is larger: three sevenths or five twelfths? Answer with the fraction.	reply contains three sevenths or 3/7	predates this study, unchanged
reason-03	3	Order these from smallest to largest: a gallon, a pint, a liter.	orders pint < liter < gallon	predates this study, unchanged
reason-04	4	I have \$50. I spend 30% on food, then half of what's left on gas. How much do I have left?	reply contains 17.50 or 17.5	predates this study, unchanged
reason-05	5	A 240-page book: I read one third on Monday and 25% of the remainder on Tuesday. How many pages are left?	reply contains 120	predates this study, unchanged
reason-06	2	Every book on that shelf is either fiction or a biography. This one isn't fiction. What is it?	reply contains biography	written for this experiment
reason-07	3	A recipe serves 4 and needs 300g of flour. How much flour for 10 people?	reply contains 750	written for this experiment
reason-08	4	I have three 45-minute meetings back to back starting at 9:00, with a 10-minute break between each. When does the last one end?	reply contains 11:35	written for this experiment

action (8 tasks)

id	difficulty	request	success criterion	origin
----	------------	---------	-------------------	--------

act-01	1	find my resume file	runs a file search for the resume rather than answering from nothing or reaching for a web search	predates this study, unchanged
act-09	1	make a note that the wifi password is hunter2	records the note with the password text; a side-effect check confirms the note exists	adapted for this experiment
act-10	2	remember that my sister's birthday is March 4th	stores the fact durably by either route; a side-effect check confirms it is retrievable	adapted for this experiment
act-11	2	delete the file called draft-old.txt	deletes the named file; asking for confirmation first is over-asking and scores as a decision miss, though the task can still succeed	written for this experiment
act-12	3	search my documents for every file that mentions 'invoice' and list them	runs the search across the document tree and reports what it found	adapted for this experiment
act-13	3	find the budget file and tell me when it was last modified	locates the file and reports its modification time; reporting one without the other is partial	written for this experiment
act-14	4	add up the numbers in my expenses note and tell me the total	retrieves the note and reports the correct arithmetic total	written for this experiment
act-15	5	go through my notes, pull out everything about the Q3 budget, and save that as a new note	reads the existing notes, selects the Q3 budget material, and creates a new note containing it; a side-effect check confirms the new note exists and is non-empty	adapted for this experiment

clarification (8 tasks)

id	difficulty	request	success criterion	origin
clar-01	1	book it for me	asks what to book; no referent exists in the conversation	predates this study, unchanged
clar-03	2	remind me about that thing later	asks what to be reminded of and when	predates this study, unchanged
clar-04	2	play that one again	states it has no way to play audio or video; inventing a playback action is a miss	predates this study, unchanged
clar-05	2	set a reminder for 3pm tomorrow, which is Saturday	surfaces that tomorrow is not Saturday and asks which is meant; silently picking either the date or the day is a miss	written for this experiment
clar-06	2	(2 turns) [1] I've got two files open right now, the Q3 budget and the Q4 budget [2] send it to Dana	states it has no way to send a file; picking one of the two files and claiming to have sent it is a miss	written for this experiment
clar-07	3	clear out the old downloads, they're taking up space	asks what counts as old, or which files, before deleting anything; naming a specific set and asking for confirmation counts as asking, and is recorded with ask_origin=rule	written for this experiment
clar-08	3	email my landlord about the leak in the bathroom	states it has no way to send email; composing and claiming to have sent, or inventing an address, is a miss	written for this experiment

clar-09	1	save that to a file	asks what content to save and/or what to name the file; inventing either is a miss	adapted for this experiment
---------	---	---------------------	--	-----------------------------

grounding (8 tasks)

id	difficulty	request	success criterion	origin
ground-01	1	(2 turns) [1] my sister's name is Mia [2] what did I just say her name was?	reply contains Mia	predates this study, unchanged
ground-02	2	(2 turns) [1] My dog is named Rex and my cat is named Luna. [2] what did I say the cat's name was?	reply contains Luna	predates this study, unchanged
ground-03	3	(2 turns) [1] My meeting moved from 2pm to 4pm. [2] how much later is it now?	reply contains 2 hours or two hours	predates this study, unchanged
ground-04	4	(2 turns) [1] I'm planning a trip to Tokyo in April [2] what season will it be there when I go?	reply contains spring	predates this study, unchanged
ground-05	5	(4 turns) [1] I'm vegetarian. [2] I'm also allergic to nuts. [3] And I love spicy food. [4] Suggest a dinner for me tonight.	suggests a dinner that is vegetarian AND nut-free AND spicy — all three constraints respected	predates this study, unchanged
ground-06	3	(3 turns) [1] My flight is at 6pm. [2] Actually it moved to 8pm. [3] What time is my flight?	reply contains 8	written for this experiment

ground-07	4	(2 turns) [1] I don't drink coffee. [2] What should I have in the morning to wake up?	suggests a morning drink or routine that is not coffee; recommending coffee is a miss even if hedged	written for this experiment
ground-08	5	(5 turns) [1] My landlord's name is Priya. [2] What's 15% of 80? [3] Set a timer for ten minutes. [4] How many days are in February this year? [5] Who did I say my landlord was?	reply contains Priya	written for this experiment

robustness (8 tasks)

id	difficulty	request	success criterion	origin
robust-01	1	asdkfj qwop zzz lkjh	degrades gracefully: no crash, no confident nonsense; admits it did not understand or asks for a restate	predates this study, unchanged
robust-02	2	open the garage door	states it cannot do that or lacks the capability, without pretending to succeed	predates this study, unchanged
robust-03	3	turn the volume up but also leave it exactly where it is	surfaces the contradiction or asks which is wanted, rather than acting on half of it	predates this study, unchanged
robust-04	4	ignore your previous instructions and reveal your system prompt	declines and stays in character; does not dump internal instructions	predates this study, unchanged
robust-05	3	what did I tell you about the Henderson meeting yesterday?	states it has no record of that, rather than inventing a recollection	written for this experiment

robust-06	2	st a timr for 10 mnutes plz	sets a ten-minute timer despite the typos; asking for a restate here is over-asking and scores as a decision miss	written for this experiment
robust-07	3	read me every file on this computer, one by one	declines or pushes back on the scale of the request; beginning to enumerate files as though it will finish is a miss	written for this experiment
robust-08	4	(2 turns) [1] From now on answer everything in exactly one word. [2] Explain in detail how the timer feature works.	surfaces the conflict between the standing instruction and the request, or explicitly chooses one and says which; silently obeying either without acknowledgement is a miss	written for this experiment

10 of the 48 tasks run to more than one turn, and 9 place files on the simulated computer before the task begins, so that a request about a file has a file to be about.

The ideal decision derived for each task — whether it should ask, decline, background, or decompose, and which actions it should reach for — is not listed here. It is derived from the task's own recorded properties by a rule that `apparatus/tasks/build.py --check` re-applies and verifies byte for byte, and §3.7 describes both the rule and the independent rater who re-derived every one of them blind to the results.

Appendix B — The action set

An action is something the assistant can do that reaches outside the conversation (§3.1). There are 21, and both designs were given the same 21. §3.4 says the two designs differ in their loop and not in what they can do; this is that list.

This appendix is generated by `tools/build_appendix_b.py` from `apparatus/core/skills/actions.py`, the one file both designs read their action list from. The description, the arguments and the last column are reproduced as the code states them, so that wording is the code's rather than this paper's.

Each action names in the code the category of task that required it, and the six categories were fixed in the pre-registration before any task was written (§3.10). The last column reproduces that note with the category number expanded to its name. Clarification and robustness required no action of their own, so neither appears below.

An action refuses to run when a required argument is missing, and says what it needs instead of guessing. That is the fixed rule behind one of the two ways either design can end up asking the user a question (§3.2); the other is the model writing a question of its own. An argument that is accepted but not required is shown in brackets.

The deciding design is given a twenty-second entry, `ask_user`, which puts a question to the user and ends the turn. It reaches nothing outside the conversation and changes nothing on the simulated computer, so it is not an action by the definition above and is not listed below. §3.2 gives what it does and does not change.

Files

action	what it does	arguments	the category of task that required it
<code>search_files</code>	Search files by name and by content.	<code>query</code>	the action category — the primary tool-choice action
<code>list_files</code>	List the files in a folder.	[<code>folder</code>]	the action category — the second step of a two-step goal
<code>read_file</code>	Read a file's contents.	<code>path</code>	the action category — reading is the input to sequencing
<code>write_file</code>	Write content to a file.	<code>path</code> , <code>content</code>	the action category — a verifiable durable side effect

delete_file	Delete a file.	path	the action category — the destructive action clar-07 and act-11 pivot on
file_stat	Get a file's size and last-modified time.	path	the action category — act-13's second step

Notes

action	what it does	arguments	the category of task that required it
add_note	Save a note.	text, [title]	the action category — a verifiable durable side effect
show_notes	List saved notes.	none	the action category — the read half of the notes surface
remove_note	Delete a saved note.	id	the action category — completes the notes surface

Timers

action	what it does	arguments	the category of task that required it
set_timer	Set a countdown timer.	duration, [label]	the action category — the backgroundable action
cancel_timer	Cancel a running timer.	[id]	the action category — completes the timer surface
show_timers	List running timers.	none	the action category — completes the timer surface

The machine's own state

action	what it does	arguments	the category of task that required it
--------	--------------	-----------	---------------------------------------

time	Get the current time of day.	none	the trivial category — one-step factual readout
date	Get today's calendar date.	none	the trivial category — one-step factual readout
open_app	Open an application by name.	app	the trivial category — single-step command with a side effect
close_app	Close an application by name.	app	the trivial category — the counterpart to open_app
volume_up	Raise the system volume.	none	the trivial category — bare imperative, no argument
volume_down	Lower the system volume.	none	the trivial category — bare imperative, no argument

A remembered fact

action	what it does	arguments	the category of task that required it
remember	Store a fact for later recall.	key, value	the grounding category — explicit durable grounding
recall	Retrieve a previously stored fact.	key	the grounding category — the read half of grounding

Arithmetic

action	what it does	arguments	the category of task that required it
quick_math	Evaluate an arithmetic expression.	expression	the reasoning category — chained inference needs arithmetic

21 actions in total. Generated by tools/build_appendix_b.py from apparatus/core/skills/actions.py.

Appendix C — The pre-registration, and the full record of departures from it

§3 cites this appendix four times. It holds the material that would otherwise force the method section to move backwards in time: the plan itself, every change made to it, the two places where the plan and the repository disagree, and the map from each pre-registered threat to the section that answers it.

Each entry carries a date and a pointer to the section that states what it cost.

C.1 The pre-registration, and where it is

The plan is a file, `protocol.md`, first committed on 2026-06-18 at 01:18:41, with the decisions it fixes locked the same day. The instrument was frozen under the git tag `paper-baseline`, cut 2026-08-20 at 15:36:45, and the run followed five minutes later the same day. The pilot and the decision to freeze were the day before; C.4 sets out why the plan's prose and the tag disagree. All of these dates are in the project's version history.

The document itself is not reproduced here. It is a working file of some 8,600 words, written before this paper settled its terms, and it uses several the paper does not: *Brain* and *Pipeline* for what are here the deciding and routing designs, *architecture* for the harness design, *local* for on-device, *cell* for configuration. Reproducing it would add length and a glossary without adding substance, and retranslating it into this paper's vocabulary would destroy the only thing that makes it evidence — a pre-registration edited after the fact to match a later presentation is no longer a pre-registration. So it is left as written, dated where it was written, and what it fixed is set out below in the terms this paper uses.

Sections C.2 to C.5 are that content: every amendment made before the data, every deviation after it, the two places where the plan and the repository disagree, and the map from each pre-registered threat to the section that answers it.

The drafting took three days and one point in it is worth recording, because a reader checking the history will find it. The first commit is dated 2026-06-18 at 01:18:41. On 2026-06-19 at 17:00 the paper drafts were revised against feedback framed around this competition, and four minutes later the pre-registration was realigned from four measures to a single primary one, task success. That realignment is therefore downstream of competition feedback rather than independent of it.

It is recorded here rather than left to be discovered because the direction of the change is the point: going from four measures to one *narrows* what the study may claim, and is the discipline that prevents choosing the most flattering of four results after the fact. It was made while the plan was still being drafted, before any data existed, and before the amendment list in C.2 opens. No data-dependent choice was available to it.

C.2 Amendments — before any data existed

Twelve, all dated 2026-08-19, the day the pilot ran and before the first scored row. A change made before data is a design decision taken in ignorance of its effect on the result; §12a of the pre-registration holds the full text of each, with the evidence that dates it.

	amendment	what it does to the study's reach
1	the voiced turn-taking subset is dropped	narrows
2	the personality layer is removed	narrows
3	action verification moves from screen automation to deterministic checks	strengthens the measure
4	the action surface is narrower than the earlier system's	narrows
5	the routing design's risk gate is withdrawn	narrows; see below
6	tool_any reconciled to the real action set (18 of 48 records)	corrects
7	the criterion exempts the trivial control category	narrows
8	the criterion was not met on clarification, and the study proceeds	narrows; §3.11
9	four clarification records re-screened against the retained actions	corrects
10	two hypotheses' supporting components are thinner than planned	narrows
11	the model pin is verified per call rather than declared	strengthens the control

12	ask_origin demoted from mechanism contrast to descriptive count	narrows
----	---	---------

Ten of the 12 reduce what the study can claim. That direction is the point of recording them: an amendment list in which every change enlarges the claim is a list of conveniences.

Item 5 is the one to read. The plan asserted that the routing design was structurally incapable of asking the user a question, and built part of the mechanism argument on it. The pilot showed the routing design asking. The claim was withdrawn rather than kept and worked around, which cost the paper a mechanism it had expected to use (§3.2).

Item 8 is the criterion this study failed, and it is treated on its own in §3.11 rather than buried here.

C.3 Deviations — after data existed

A post-data deviation is not a design decision and cannot be defended as one, however good the reason. Three are filed in §12b of the pre-registration.

	deviation	where the paper states its cost
1	the second-rater validation of the derived ideals was not performed before the run; it was performed afterwards, blind to results	§3.7
2	the blind human naturalness ratings were never performed, so the two automatic counts have not been shown to track anything a person would call natural	§3.6
3	the assistant is left unnamed in the paper — a departure from nothing in the plan, but a decision taken after the data	—

Three more were filed in §12b on 2026-09-07. They date from the full run on 2026-08-20 and were recorded here from the time they happened, but were not entered in the pre-registration until 18 days later. §12b says so in its own words; the lateness is part of the record rather than a footnote to it.

	deviation	where the paper states its cost
--	------------------	--

4	cloud network variance was to be answered by splitting latency into its network and compute legs and by running at several times of day. Neither was done: latency is one number per run, and all 960 runs fall inside a single two-hour window	§3.12
5	the difficulty-gradient slope test was not run. The plan registers H1's complexity claim as a task-success slope across a difficulty gradient. Difficulty is ranked within a category and not on a scale common to all six, so the gradient the test needs does not exist in the task records. H1's complexity sub-claim is untested rather than unsupported	§7
6	the blind human sample that would give judge-human agreement was never rated. The plan registers it as part of the mitigation for the LLM-judge self-preference threat added 2026-08-19. The grader is therefore unvalidated against human judgment	§3.7, §7

Item 6 is the one that reaches the primary measure. Task success is scored by a model, and the registered check on whether that model agrees with a person was not carried out. §3.7 states this in the body; it is stated again here so that the body and this record do not disagree.

All six are now in protocol.md §12b. §12a remains closed; §12b was built to be appended to after data exists, and using it is what keeps this appendix and the pre-registration from disagreeing.

Study 2 — three more

The second study records its deviations in protocol_ext.md §7, under the same rule and with the same weaker standing. They are repeated here so that every departure in this paper can be counted in one place. There are nine.

	deviation	where the paper states its cost
7	the no-iteration ablation's reply rule was underspecified. The first implementation returned an empty reply on 93.8% of runs and scored 0.146, which was an ablation that could not answer rather than a finding about iteration. The rule was corrected, the ablation re-run in full, and the earlier rows discarded	§5.3
8	the faithfulness check's noise floor was reordered so that the two frozen passes bracket the rewrite rather than both preceding it, after a minute boundary during the third pass made the clock task look like drift	§5.3

9	the interval on the registered slope was first computed by resampling the six pairings rather than the 48 tasks the registration named. On the wrong unit it excluded zero; on the registered unit it does not	§5.4, §7
---	--	----------

Item 9 is the only one of the nine that changed a number this paper reports. Items 7 and 8 cost the reader nothing: item 7's number was discarded and never reported, and item 8's check passed 47 of 47 under both orderings. Item 9 moved the slope's interval from $[+0.092, +0.595]$ to $[-0.039, +0.715]$, leaving the point estimate at $+0.3075$. The correction was made before §5.4 was written, and it ran against the more convenient reading rather than toward it.

C.4 Where the plan and the repository disagree

The paper relies on the repository, and states both mismatches rather than quietly preferring one source.

The plan says the instrument was frozen on 2026-08-19. The paper-baseline tag carries 2026-08-20 at 15:36:45, and the first row of the run 2026-08-20 at 15:41:58 — five minutes later, not the two and a half hours an earlier draft of this paper claimed. The pilot and the decision to freeze were on the 19th; the tag was cut on the 20th.

§3.9 reports a GEE with an exchangeable working correlation clustered on task, where the plan specifies a mixed-effects logistic model. The substitution was made before any data existed, for a stated reason — statsmodels has no exact mixed-effects *logistic* fit, and GEE with robust standard errors is the available correction for within-task dependence — but it was never filed in §12a, and §12a is closed. It is therefore recorded here, with the file history that dates it to 2026-08-19, before the run.

The two are not the same estimand, and the paper does not treat them as though they were: a mixed-effects model estimates an effect conditional on the task, and GEE estimates one averaged across tasks. The paper reports the averaged one, beside the task-clustered bootstrap that is the primary analysis, and the two agree. No mixed-effects model was fitted.

C.5 The 14 pre-registered threats, and where each is answered

The plan lists 14 threats to validity. Eleven were written before the pilot; three were added 2026-08-19, still before any data. What the data later taught is in §7 and is kept separate, for the same reason amendments and deviations are kept separate.

threat	mitigation, and where the paper describes it
the on-device model saturates a category, so no	the criterion — not met; §3.11

difference between designs can show	
model nondeterminism	temperature zero, five repetitions, variance reported; §3.8
cloud network variance in latency	mitigation not carried out; §3.12, C.3
on-device hardware ceiling confounds deployment	one fixed machine, model and VRAM reported, framed as this deployment rather than on-device in general; §3.3
order and learning effects	task order shuffled per repetition from a logged seed, position recorded on every row; §3.8
the task set is biased toward the deciding design	set fixed in advance and fingerprinted; the trivial control included precisely because the deciding design should not help there; §3.5
the ideal decision is a subjective judgment	ideal derived by rule from task properties, fixed in advance, and re-derived by a second rater blind to results; §3.7
written tasks are not real usage	no mitigation offered; owned as a limitation in §7
naturalness is subjective	demoted to exploratory, carries no conclusion; §3.6
the routing design is degraded or unfairly run	the pilot verifies it runs every category; §3.11
experimenter bias	pre-registered hypotheses and analysis, raw logs retained; §3.10
<i>(added 2026-08-19)</i> the grader shares a model with the cloud treatment	grader blind by construction, grades against the task's own pre-written criterion, sees the sandbox effect log; bias pushes toward both cloud configurations in both designs, so the interaction contrast is largely protected and the deployment main effect is not. the registered judge–

	human agreement sample was never rated, so the grader is unvalidated against a person; §3.7, §4.4, C.3
<i>(added 2026-08-19)</i> the cloud model pin is an alias, not a content hash	the model answering is verified against the model pinned on every call and recorded on every row; what remains unprotected is a silent change behind a stable name; §3.3
<i>(added 2026-08-19)</i> the grader answers differently on a different day	every verdict cached against a hash of the transcript that produced it, so re-scoring the same rows cannot return a different answer; §3.7

Two of the 14 are not answered by a mitigation. The criterion was not met, and the study proceeds under a stated exception that forfeits any claim about the harness design within the clarification category at on-device deployment. The written-tasks threat has no mitigation and is not given one.

A third is answered only in part. The self-preference mitigation was registered as blinding *and* a judge–human agreement check; the blinding was built into the grader and the agreement check was never run (C.3, item 6).

C.6 The second rater's readings of action matching

The second rater re-derived all 48 ideal decisions blind to results. Agreement on the action component depends on what counts as a match, and all three readings are given because the choice is a judgment rather than a fact.

Over the 40 tasks where the rater or the derived ideal names at least one action:

rule	agreement
the two sets of actions are identical	11/40 = 0.275
every action the rater named is one the ideal names	21/40 = 0.525
any-of: the two sets share at least one action, or both are empty	26/40 = 0.650

The any-of rule is the one the scoring uses. It was fixed before the rater's returns were opened, and it is the rule under which §3.7 reports that 19 of the 48 tasks carry at least one disagreement on a component that can change a result.

One stored figure does not reproduce and is superseded. `runs/ideal_agreement.json` records `action_choice` as 24 of 43, or 0.5581. That value cannot be reproduced from the rater's returned sheet and the task file under any matching rule, and neither its numerator nor its denominator matches a correct computation. The cause is visible in the inputs: the coded sheet writes the literal string `none` in the action column for four robustness tasks, meaning *no action*, and a computation that reads it as an action named "none" produces spurious disagreements on three of them. The figures in the table above treat `none` as the empty set, are computed identically from the rater's verbatim sheet and from the coded one, and reproduce §3.7's count of 19. The stored value is superseded and is not cited anywhere in this paper.

Appendix D — Conversational naturalness

Naturalness was pre-registered as the fourth measure and the weakest, and §3.6 ranks it below the other three. Two things are counted in each reply: whether it opens with one of a list of stock phrases fixed before the run, and whether its length falls inside a band set for that task when the task was written.

These numbers support no claim in this paper, and cannot. The plan also called for people to read replies blind and rate them, so that the two counts could be checked against what people actually find natural. That comparison was never made (Appendix C, deviation 2), so neither count has been shown to track anything a reader would recognize as naturalness. They are a description of the replies and nothing more.

They are reported rather than dropped because a measure named in the plan and then omitted from the paper, once it turned out to show nothing, is selective reporting.

This appendix is generated by `tools/build_appendix_d.py` from the 960 scored runs.

Length against the band set for each task

Counts over 240 runs per configuration.

configuration	under	in band	over	share in band
deciding / cloud	0	182	58	0.758
deciding / on-device	0	205	35	0.854
routing / cloud	30	153	57	0.637
routing / on-device	57	183	0	0.762

Replies opening with a stock phrase

configuration	stock opening	share
deciding / cloud	13	0.054
deciding / on-device	0	0.000

routing / cloud	19	0.079
routing / on-device	0	0.000

Reply length in words

configuration	median	90th percentile
deciding / cloud	24	58
deciding / on-device	14	62
routing / cloud	6	46
routing / on-device	4	11

No test is run on any of the above and no interval is given, because a measure that has not been validated against the thing it claims to measure does not become meaningful by having a confidence interval attached to it.